

In [1]: `import numpy as np`

In [2]: `import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
#plt.style.use('fivethirtyeight')
import warnings
warnings.filterwarnings('ignore')`

In [3]: `iris=pd.read_csv(r'C:\Users\DELL\Downloads\24th, 25th- Advanced EDA project\IRIS`

In [4]: `iris`

Out[4]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa
...	...	...	...	...	...	...
145	146	6.7	3.0	5.2	2.3	Iris-virginica
146	147	6.3	2.5	5.0	1.9	Iris-virginica
147	148	6.5	3.0	5.2	2.0	Iris-virginica
148	149	6.2	3.4	5.4	2.3	Iris-virginica
149	150	5.9	3.0	5.1	1.8	Iris-virginica

150 rows × 6 columns

In [5]: `iris.head()`

```
Out[5]:
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

```
In [6]: iris.drop('Id',axis=1,inplace=True)
```

```
In [7]: iris.head()
```

```
Out[7]:
```

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

```
In [8]: iris.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   SepalLengthCm    150 non-null    float64
1   SepalWidthCm     150 non-null    float64
2   PetalLengthCm    150 non-null    float64
3   PetalWidthCm     150 non-null    float64
4   Species          150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

```
In [9]: iris['Species'].value_counts()
```

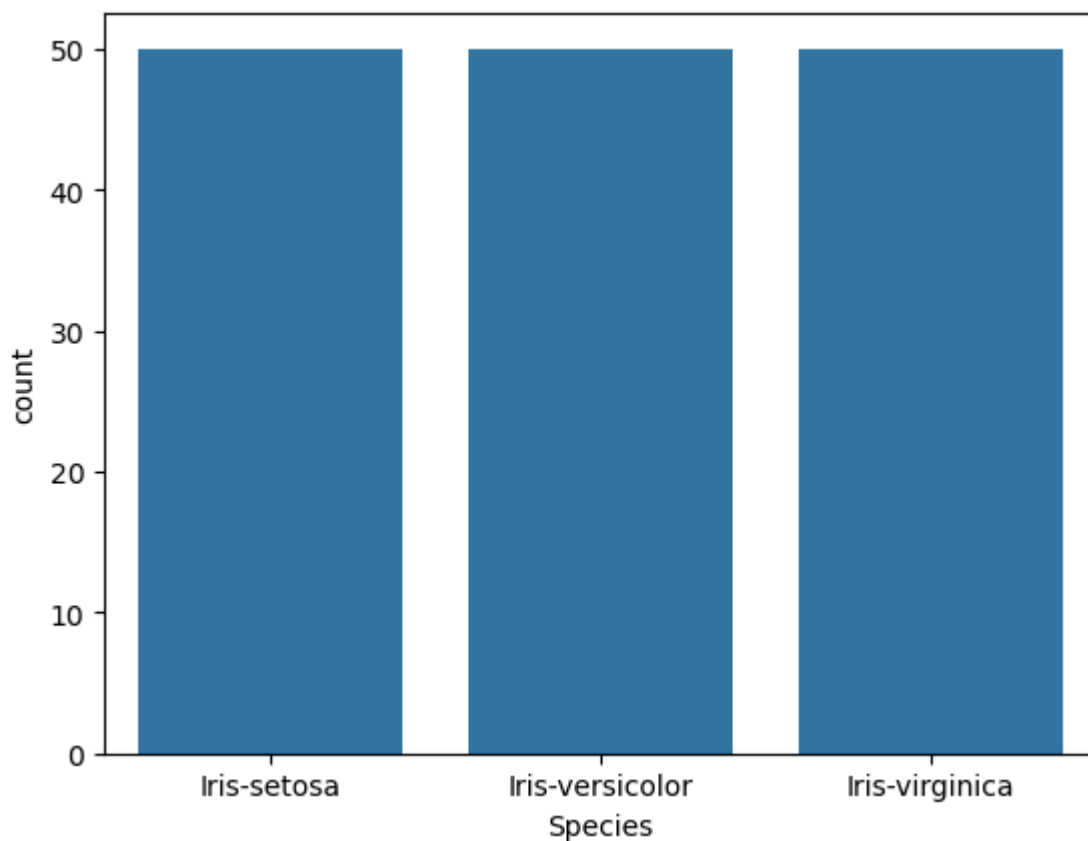
```
Out[9]: Species
Iris-setosa      50
Iris-versicolor  50
Iris-virginica   50
Name: count, dtype: int64
```

Bar Plot

```
In [11]: sns.countplot('Species',data=iris)
plt.show()
```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[11], line 1  
----> 1 sns.countplot('Species',data=iris)  
      2 plt.show()  
  
TypeError: countplot() got multiple values for argument 'data'
```

```
In [12]: sns.countplot(x='Species', data=iris)  
         plt.show()
```



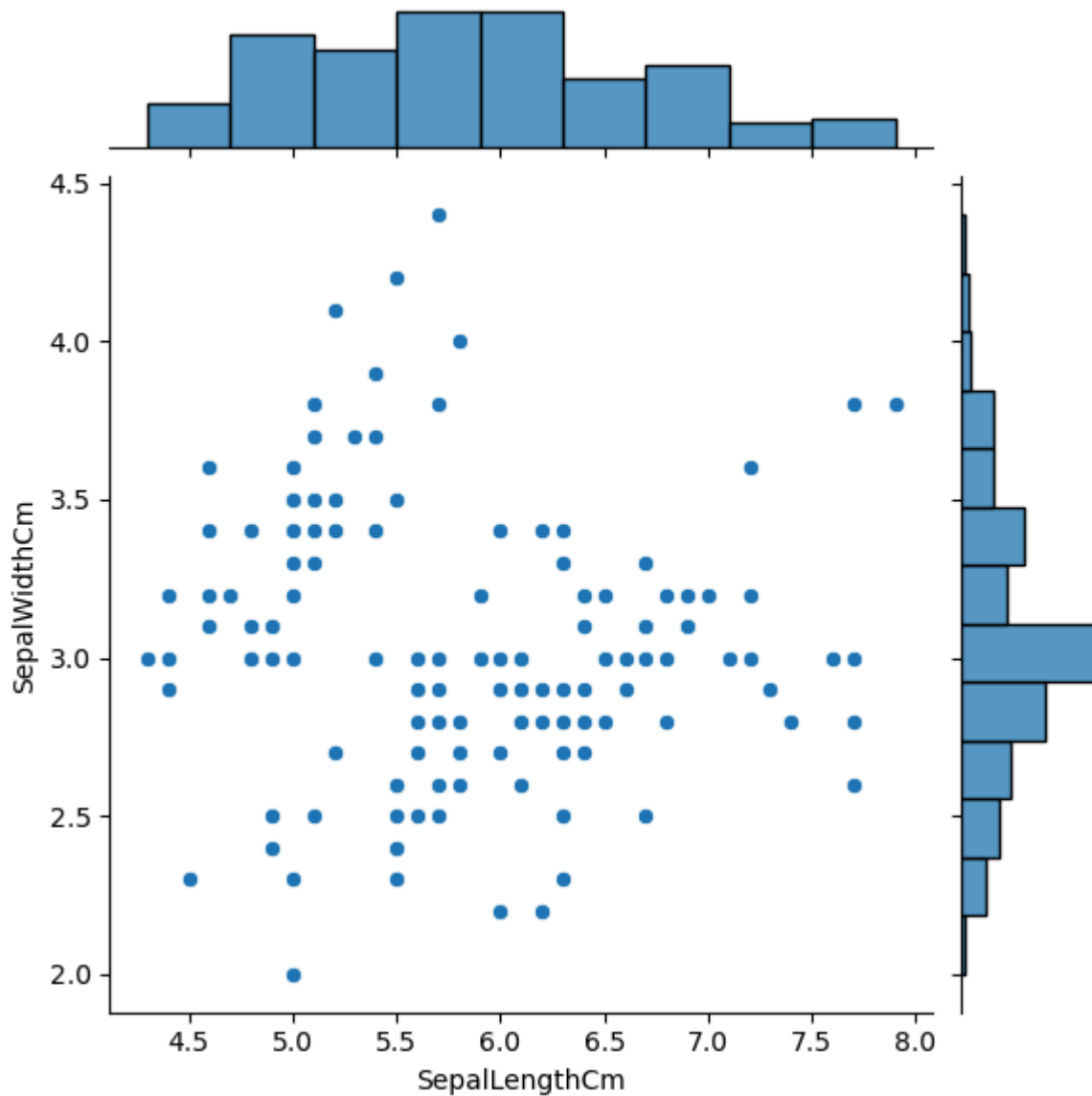
Joint Plot

```
In [13]: iris.head()
```

```
Out[13]:
```

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

```
In [14]: fig=sns.jointplot(x='SepalLengthCm',y='SepalWidthCm',data=iris)
```

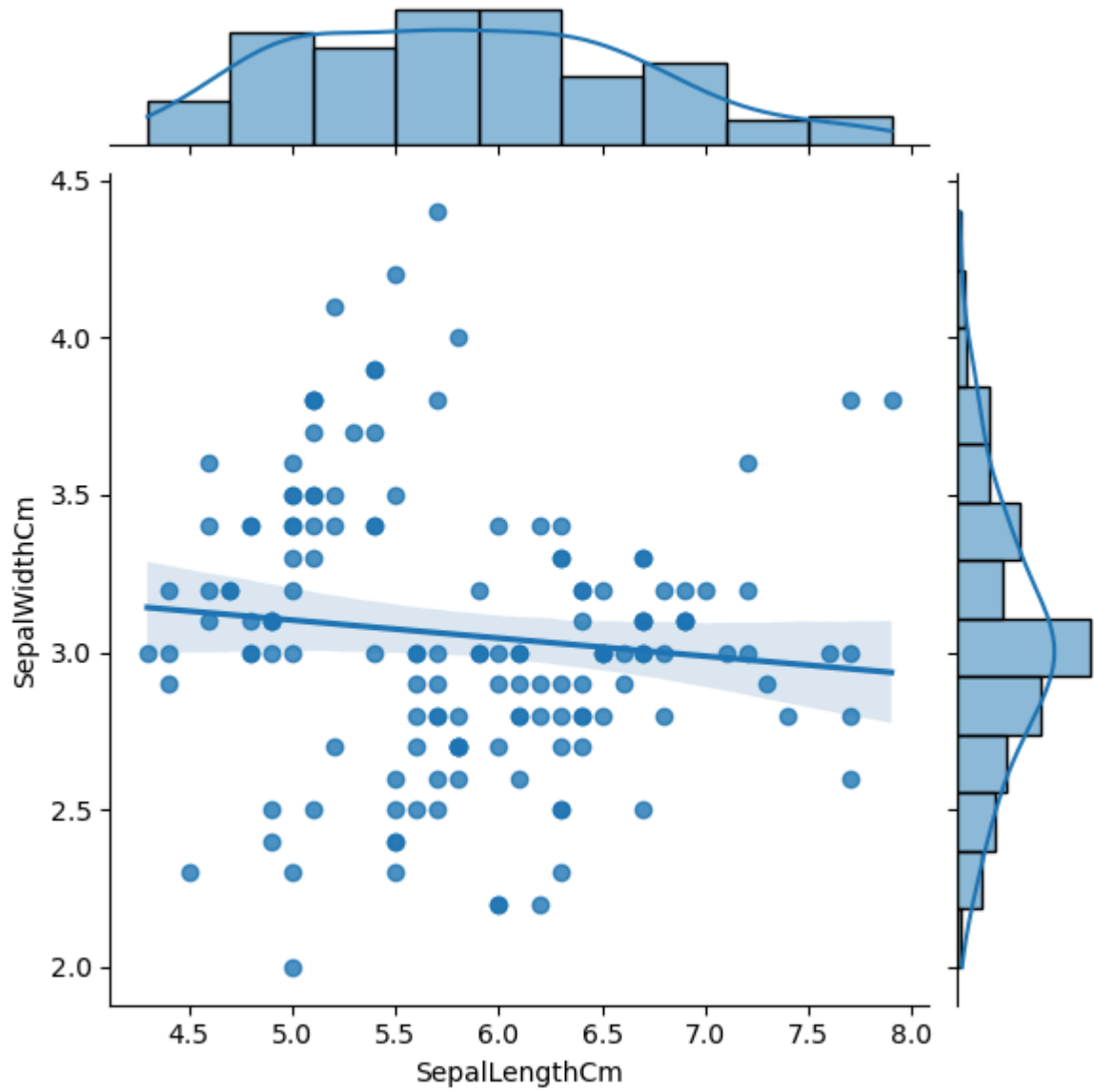


```
In [15]: sns.jointplot("SepalLengthCm", "SepalWidthCm", data=iris, kind="reg")
```

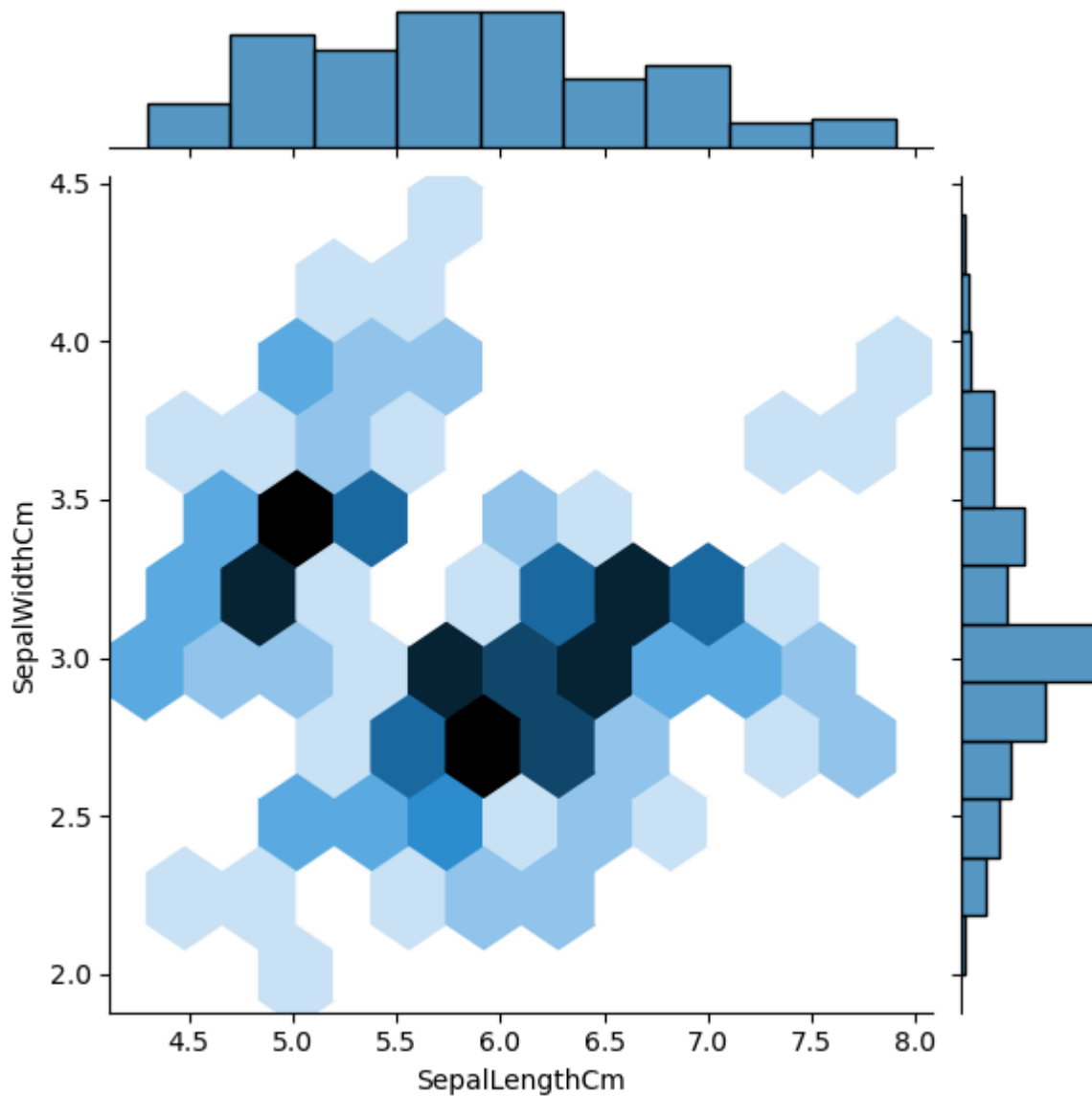
```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[15], line 1  
----> 1 sns.jointplot("SepalLengthCm", "SepalWidthCm", data=iris, kind="reg")  
  
TypeError: jointplot() got multiple values for argument 'data'
```

```
In [16]: sns.jointplot(x="SepalLengthCm",y="SepalWidthCm", data=iris, kind="reg")
```

```
Out[16]: <seaborn.axisgrid.JointGrid at 0x1a830a6d7c0>
```



```
In [17]: fig=sns.jointplot(x='SepalLengthCm',y='SepalWidthCm',kind='hex',data=iris)
```



Facet Grid plot

```
In [18]: import matplotlib.pyplot as plt
%matplotlib inline

sns.FacetGrid(iris,hue='Species',size=5)\
.map(plt.scatter,'SepalLengthCm','SepalWidthCm')\
.add_legend()
```

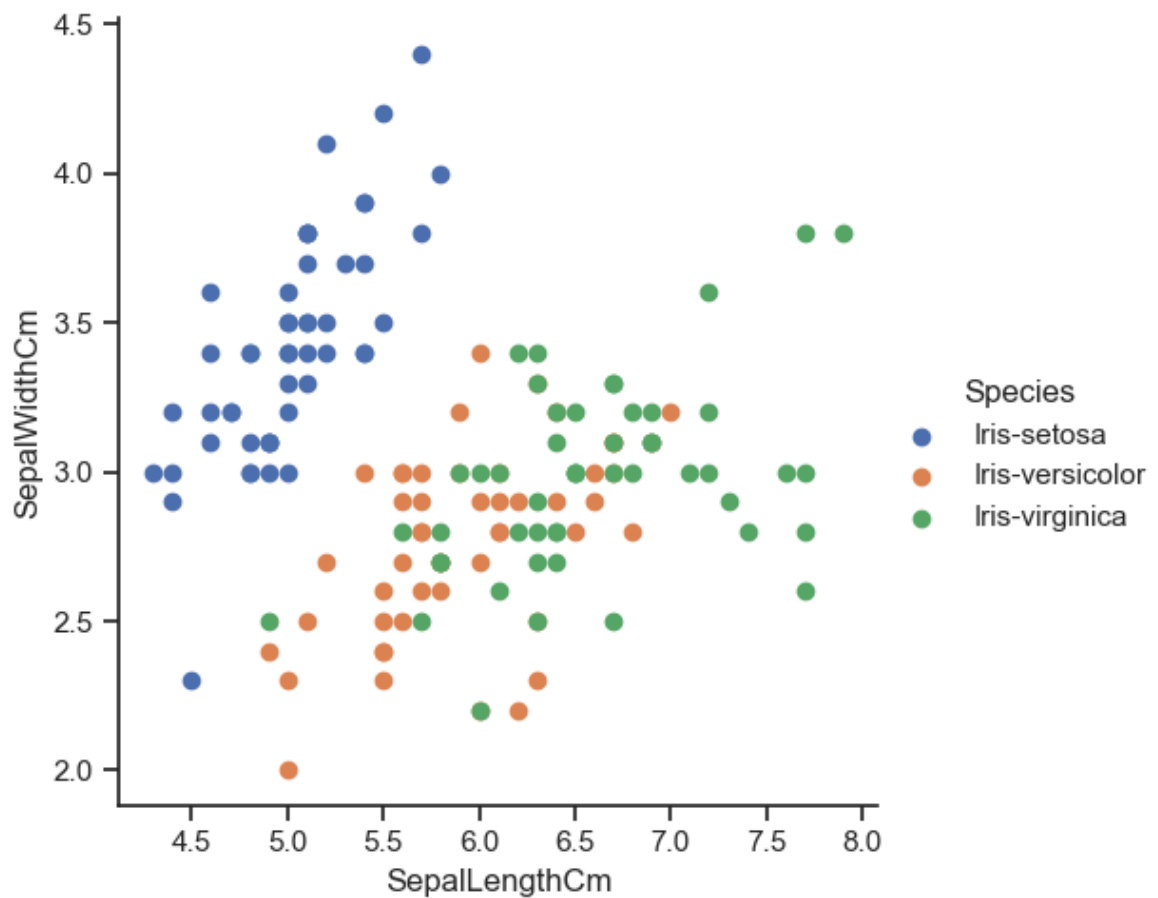
```
-----
TypeError                                Traceback (most recent call last)
Cell In[18], line 4
      1 import matplotlib.pyplot as plt
      2 get_ipython().run_line_magic('matplotlib', 'inline')
----> 4 sns.FacetGrid(iris,hue='Species',size=5)\
      5 .map(plt.scatter,'SepalLengthCm','SepalWidthCm')\
      6 .add_legend()

TypeError: FacetGrid.__init__() got an unexpected keyword argument 'size'
```

```
In [19]: import seaborn as sns
import matplotlib.pyplot as plt

sns.set(style="ticks")
```

```
g = sns.FacetGrid(iris, hue='Species', height=5)
g.map(plt.scatter, 'SepalLengthCm', 'SepalWidthCm')
g.add_legend()
plt.show()
```



Boxplot or Whisker plot

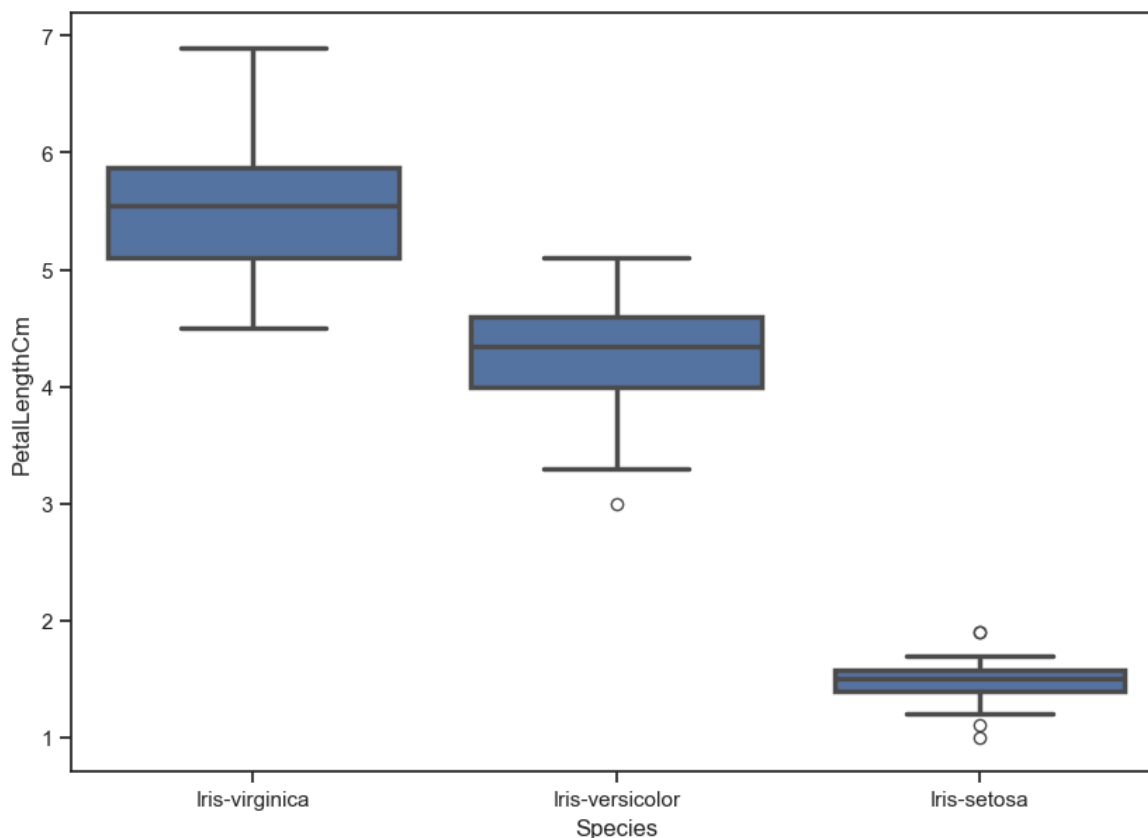
```
In [20]: iris.head()
```

```
Out[20]:
```

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

```
In [21]: fig=plt.gcf()
fig.set_size_inches(10,7)
fig=sns.boxplot(x='Species',y='PetalLengthCm',data=iris,order=['Iris-virginica',
```

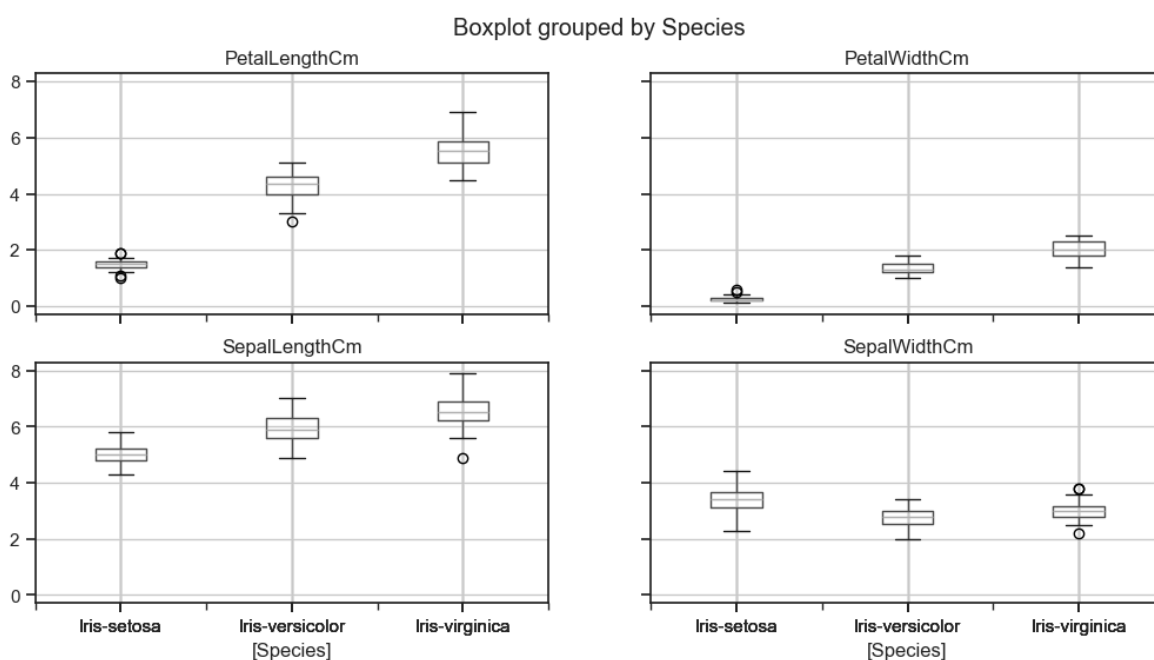
```
In [22]: plt.show()
```



```
In [23]: iris.boxplot(by="Species", figsize=(12, 6))
```

```
Out[23]: array([[<Axes: title={'center': 'PetalLengthCm'}, xlabel='[Species]>',
  <Axes: title={'center': 'PetalWidthCm'}, xlabel='[Species]>'],
  [<Axes: title={'center': 'SepalLengthCm'}, xlabel='[Species]>',
  <Axes: title={'center': 'SepalWidthCm'}, xlabel='[Species]>']],
  dtype=object)
```

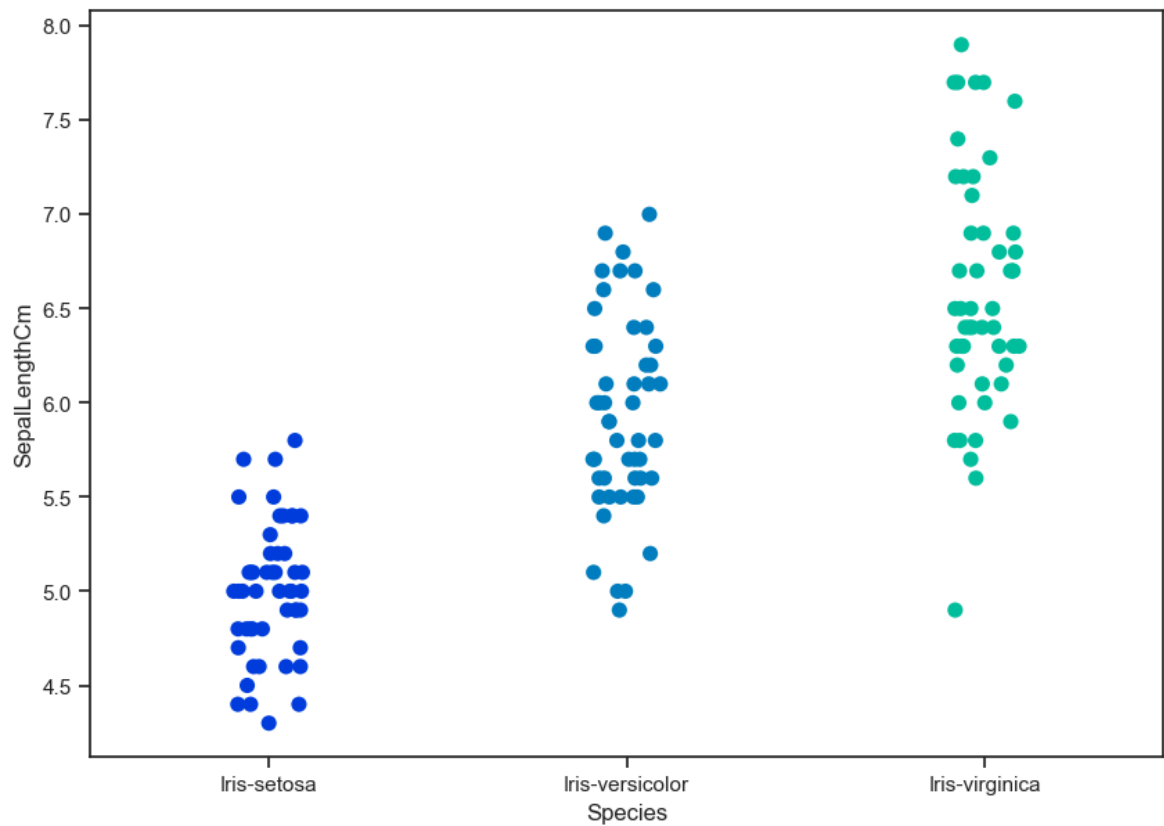
```
In [24]: plt.show()
```



Strip Plot


```
In [25]: fig=plt.gcf()
fig.set_size_inches(10,7)
fig=sns.stripplot(x='Species',y='SepalLengthCm',data=iris,jitter=True,edgecolor=

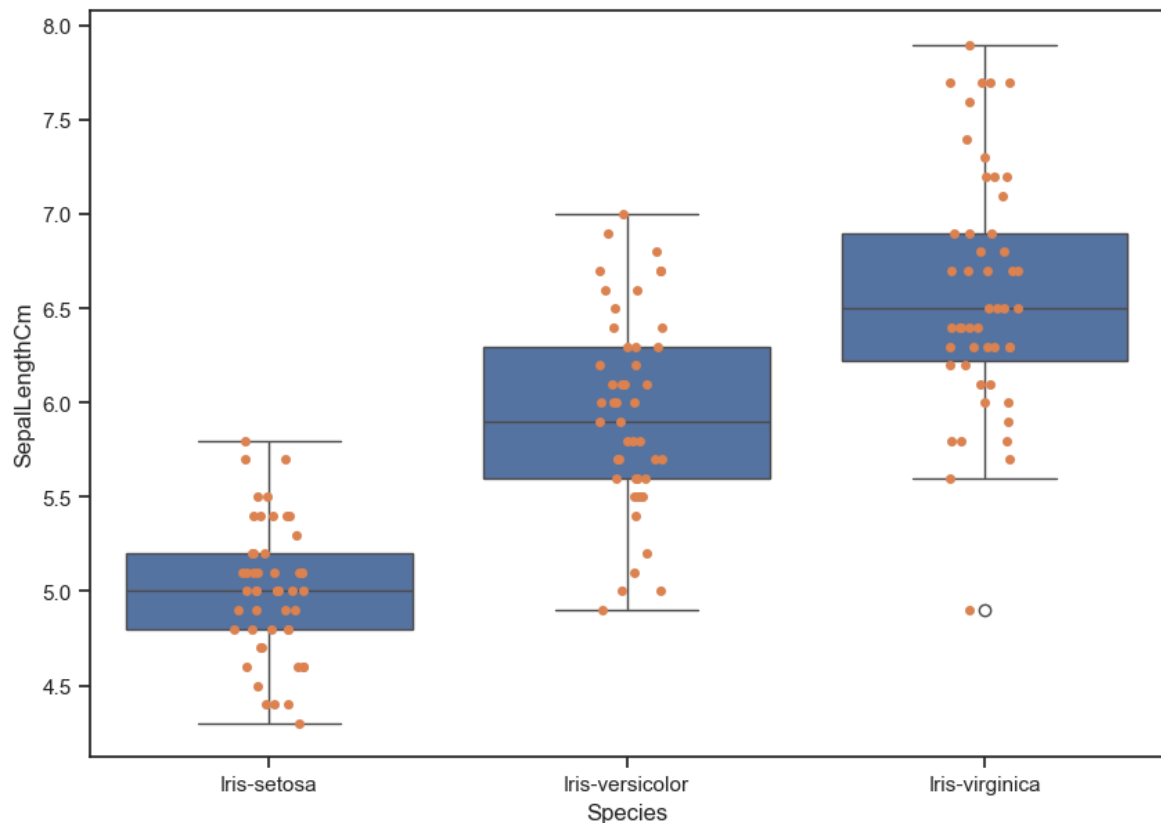
In [26]: plt.show()
```



Combining Box and Strip plots

```
In [27]: fig=plt.gcf()
fig.set_size_inches(10,7)
fig=sns.boxplot(x='Species',y='SepalLengthCm',data=iris)
fig=sns.stripplot(x='Species',y='SepalLengthCm',data=iris,jitter=True,edgecolor=

In [28]: plt.show()
```



```
In [29]: ax = sns.boxplot(x="Species", y="PetalLengthCm", data=iris)
ax = sns.stripplot(x="Species", y="PetalLengthCm", data=iris, jitter=True, edgecolor="gray")

boxtwo = ax.artists[2]
boxtwo.set_facecolor('yellow')
boxtwo.set_edgecolor('black')
boxthree=ax.artists[1]
boxthree.set_facecolor('red')
boxthree.set_edgecolor('black')
boxthree=ax.artists[0]
boxthree.set_facecolor('green')
boxthree.set_edgecolor('black')

plt.show()
```

```
-----
IndexError                                Traceback (most recent call last)
Cell In[29], line 4
      1 ax= sns.boxplot(x="Species", y="PetalLengthCm", data=iris)
      2 ax= sns.stripplot(x="Species", y="PetalLengthCm", data=iris, jitter=True,
edgecolor="gray")
----> 4 boxtwo = ax.artists[2]
      5 boxtwo.set_facecolor('yellow')
      6 boxtwo.set_edgecolor('black')

File ~\anaconda3\Lib\site-packages\matplotlib\axes\_base.py:1453, in _AxesBase.ArtistList.__getitem__(self, key)
    1452 def __getitem__(self, key):
-> 1453     return [artist
    1454               for artist in self._axes._children
    1455               if self._type_check(artist)][key]

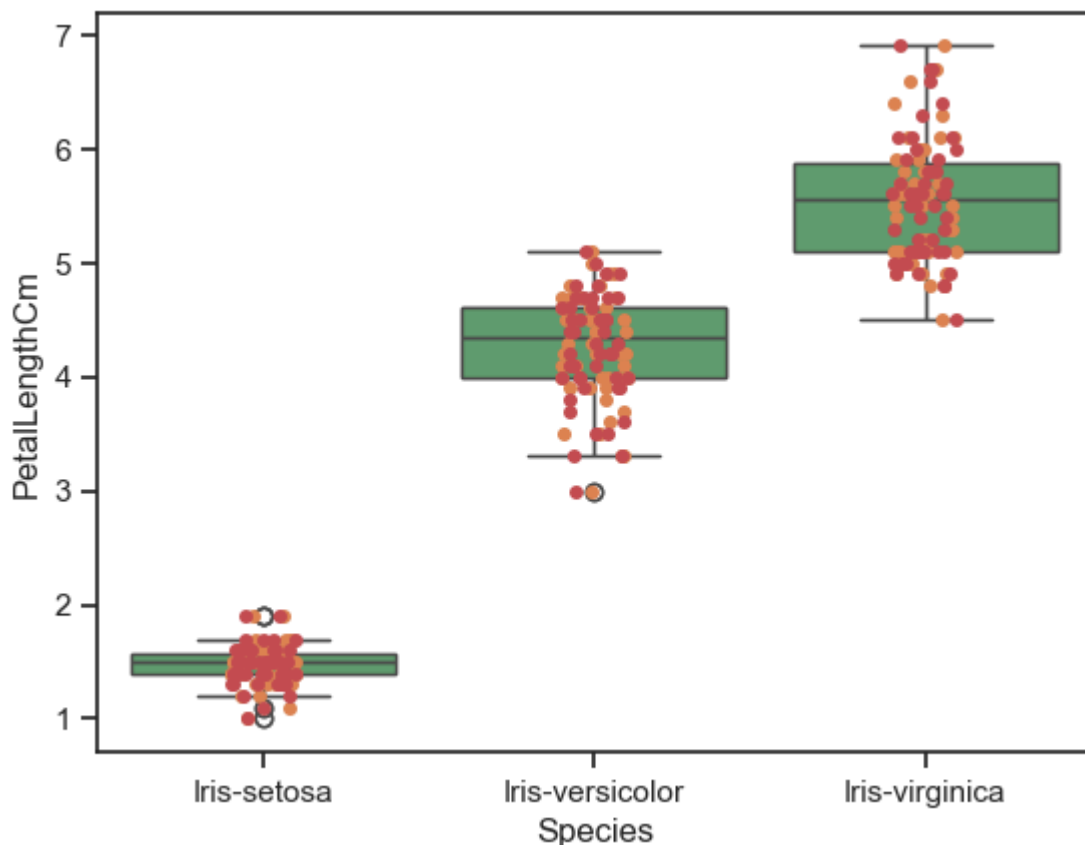
IndexError: list index out of range
```

```
In [30]: import seaborn as sns
import matplotlib.pyplot as plt

ax = sns.boxplot(x="Species", y="PetalLengthCm", data=iris)
sns.stripplot(x="Species", y="PetalLengthCm", data=iris, jitter=True, edgecolor=

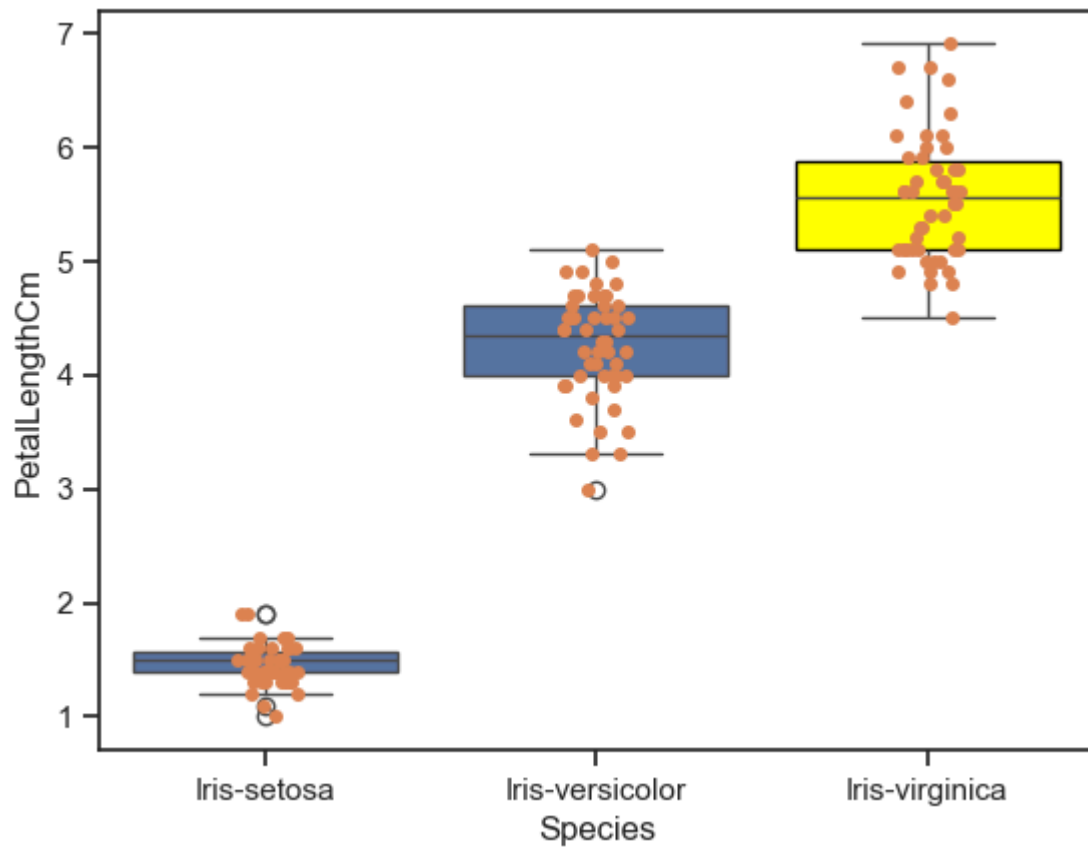
# Access the box elements from ax.patches
# Usually, there's one box per category (species)
# Let's change the second box (index 1)
box = ax.patches[1]
box.set_facecolor('yellow')
box.set_edgecolor('black')

plt.show()
```



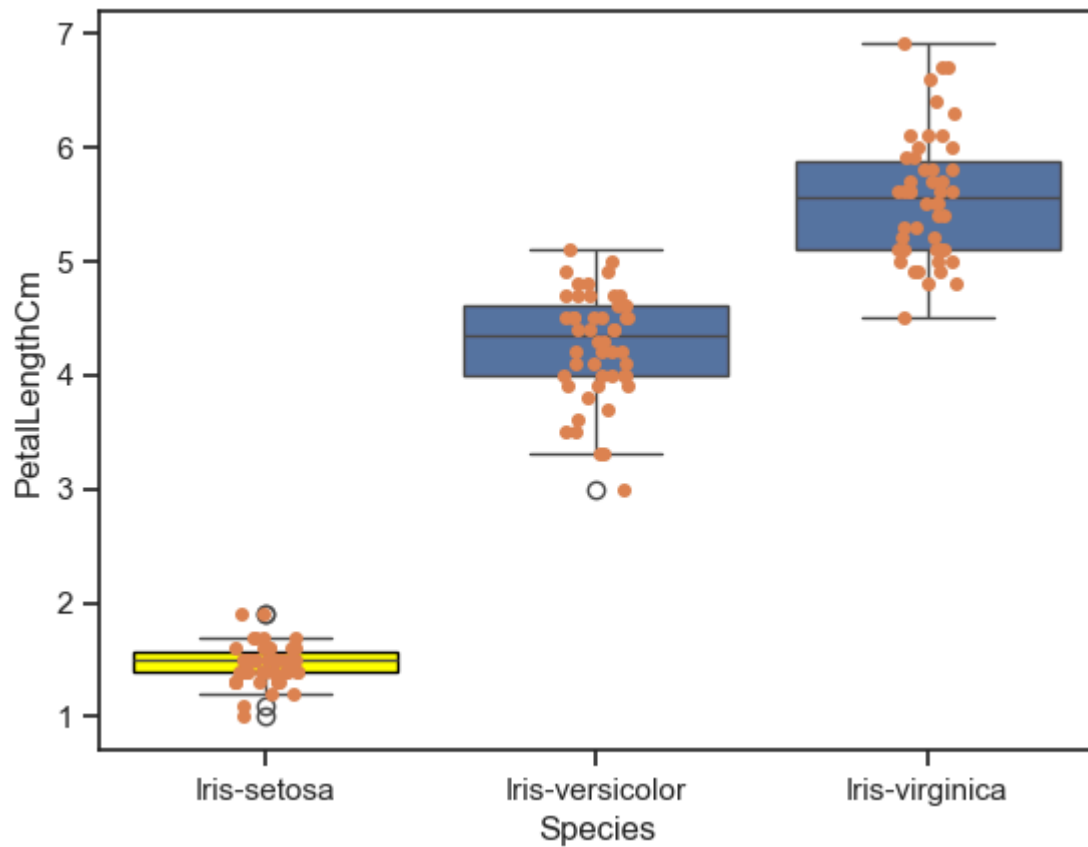
```
In [31]: ax= sns.boxplot(x="Species", y="PetalLengthCm", data=iris)
ax= sns.stripplot(x="Species", y="PetalLengthCm", data=iris, jitter=True, edgeco
box = ax.patches[2]
box.set_facecolor('yellow')
box.set_edgecolor('black')

plt.show()
```



```
In [32]: ax = sns.boxplot(x="Species", y="PetalLengthCm", data=iris)
ax = sns.stripplot(x="Species", y="PetalLengthCm", data=iris, jitter=True, edgecolor='black')
box = ax.patches[0]
box.set_facecolor('yellow')
box.set_edgecolor('black')

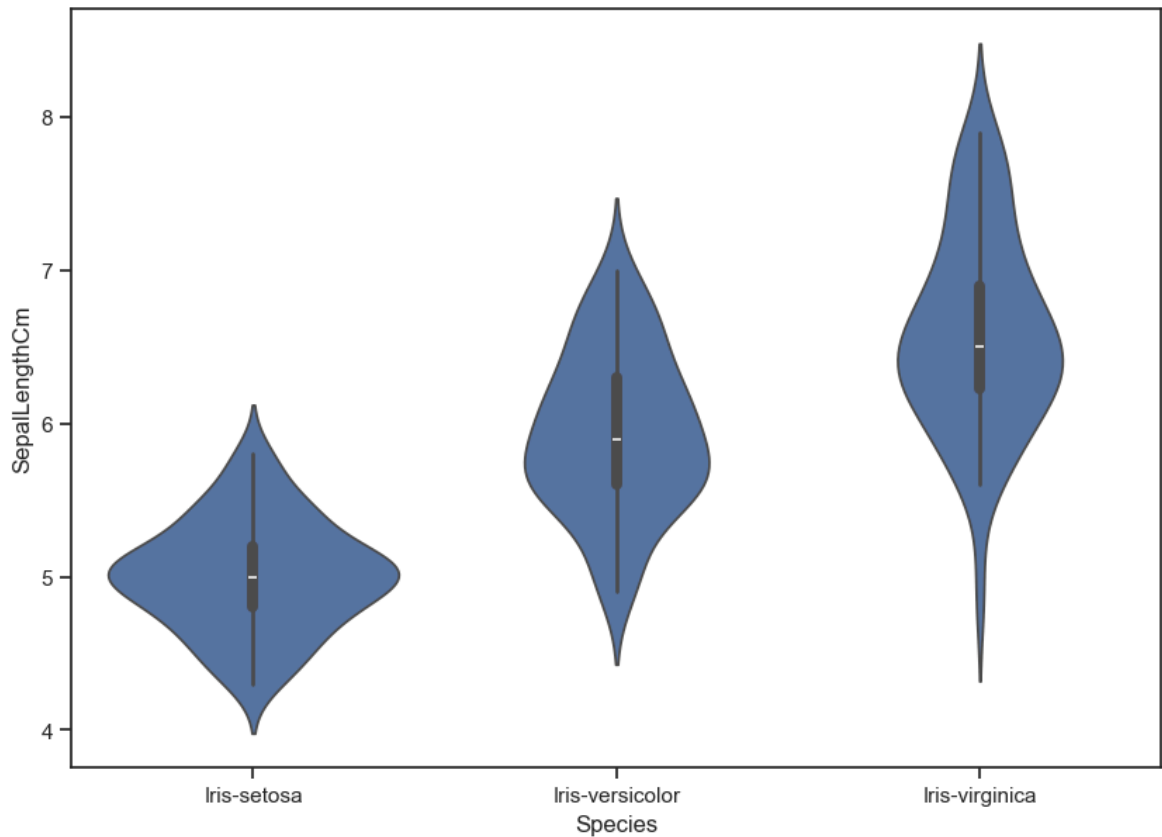
plt.show()
```



Violin plot

```
In [33]: fig=plt.gcf()
fig.set_size_inches(10,7)
fig=sns.violinplot(x='Species',y='SepalLengthCm',data=iris)
```

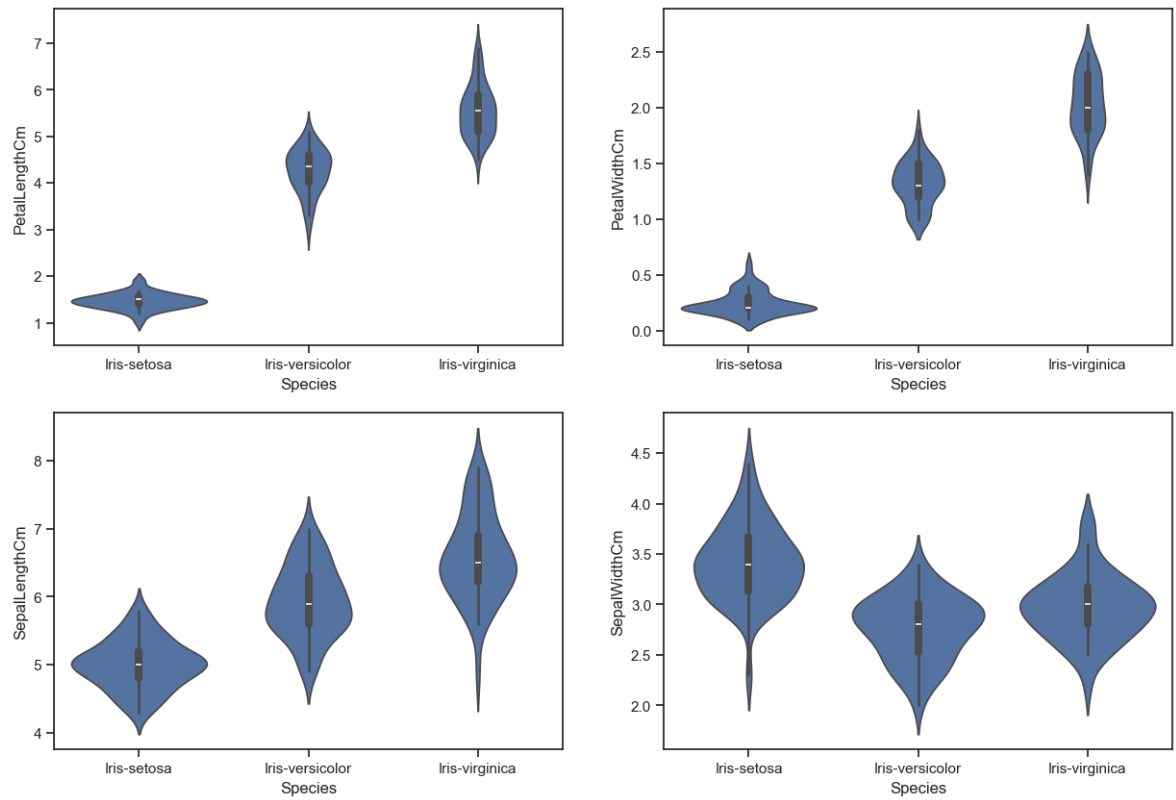
```
In [34]: plt.show()
```



```
In [35]: plt.figure(figsize=(15,10))
plt.subplot(2,2,1)
sns.violinplot(x='Species',y='PetalLengthCm',data=iris)
plt.subplot(2,2,2)
sns.violinplot(x='Species',y='PetalWidthCm',data=iris)
plt.subplot(2,2,3)
sns.violinplot(x='Species',y='SepalLengthCm',data=iris)
plt.subplot(2,2,4)
sns.violinplot(x='Species',y='SepalWidthCm',data=iris)
```

```
Out[35]: <Axes: xlabel='Species', ylabel='SepalWidthCm'>
```

```
In [36]: plt.show()
```

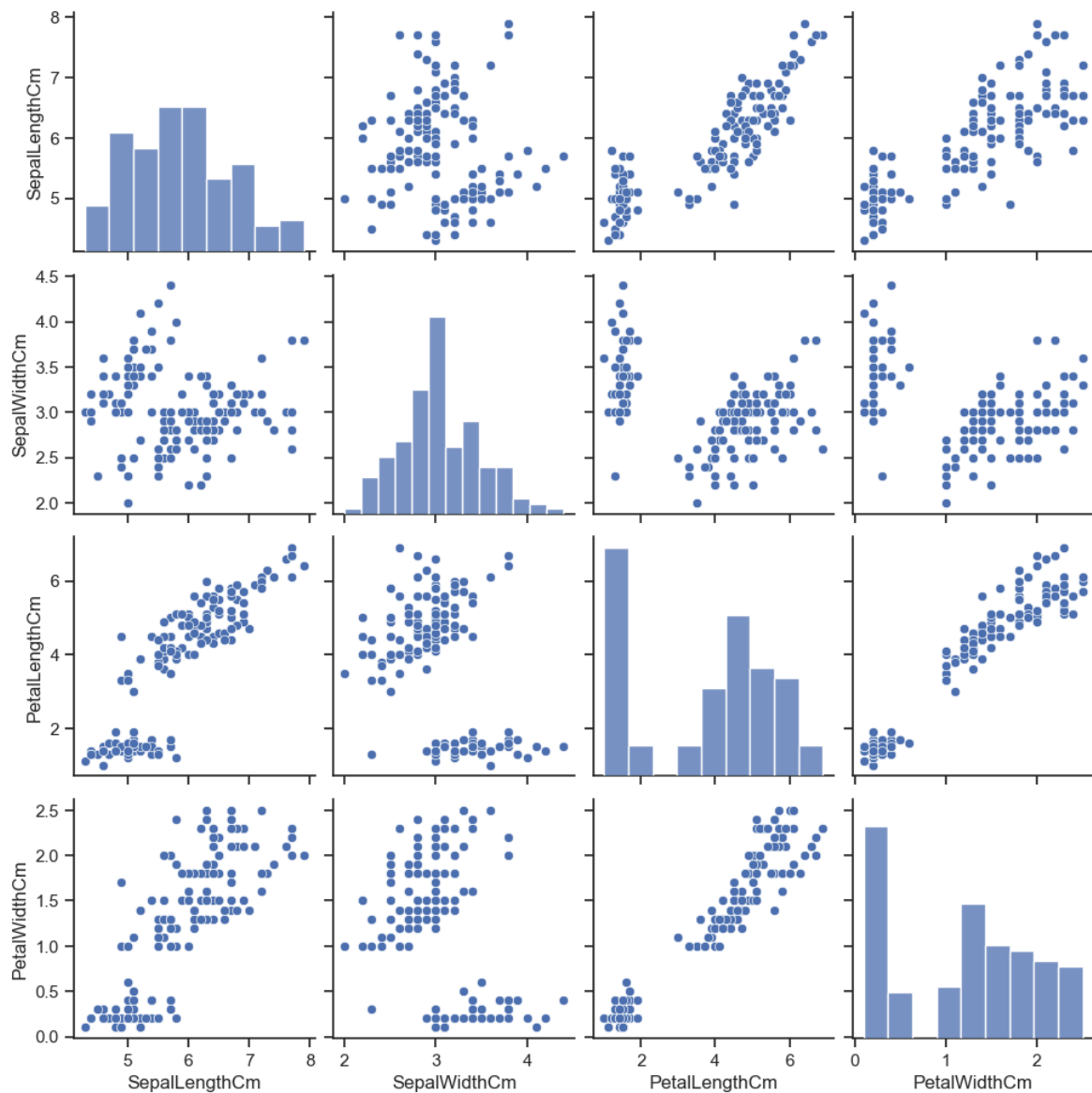


pair plot

```
In [37]: sns.pairplot(data=iris,kind='scatter')
```

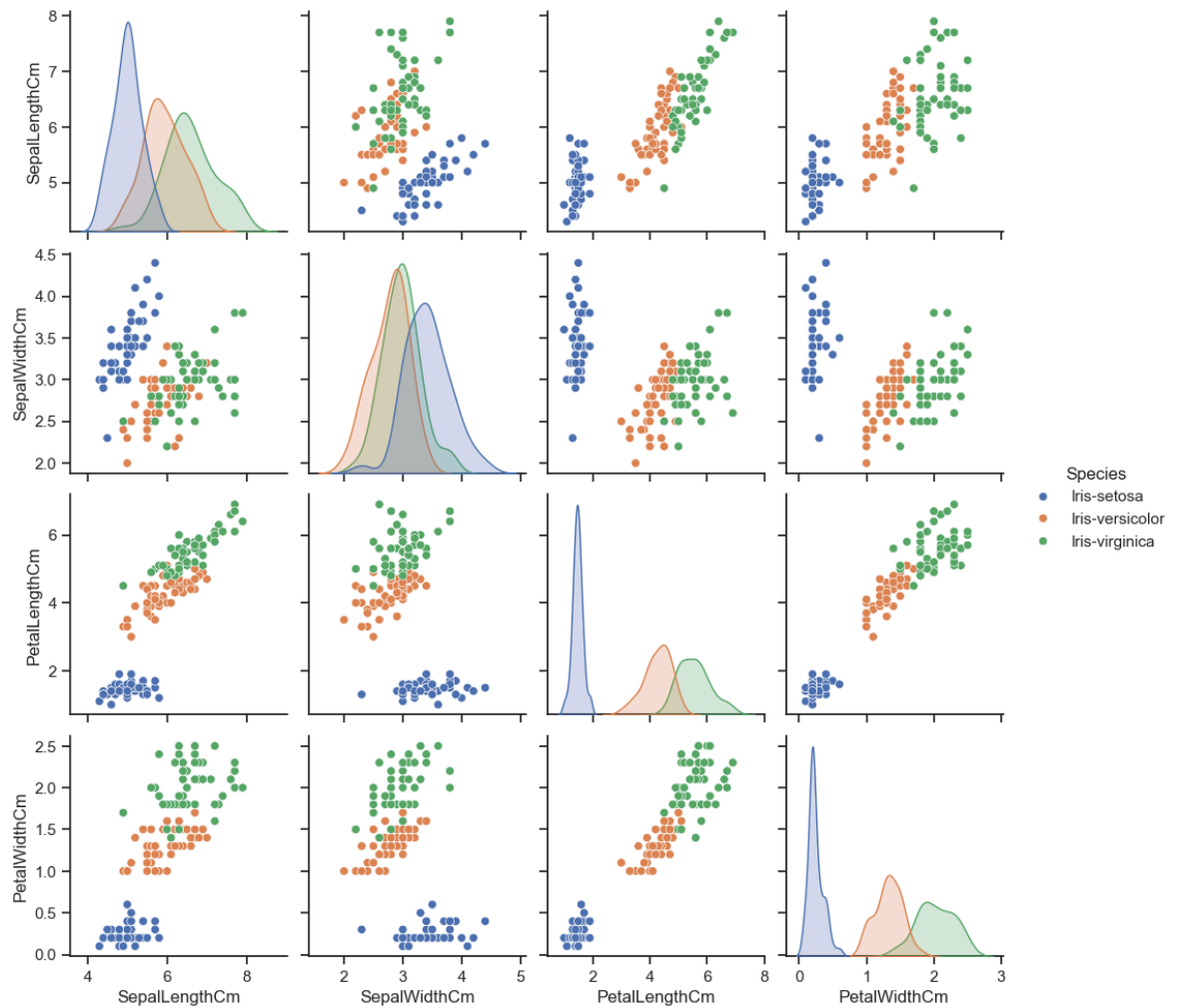
```
Out[37]: <seaborn.axisgrid.PairGrid at 0x1a836cdf1d0>
```

```
In [38]: plt.show()
```



```
In [39]: sns.pairplot(iris,hue='Species');
```

```
In [40]: plt.show()
```

Heat Map

```
In [41]: fig=plt.gcf()
fig.set_size_inches(10,7)
fig=sns.heatmap(iris.corr(),annot=True,cmap='cubehelix',linewidths=1,linewidthcolor='')
```

```

-----
ValueError                                Traceback (most recent call last)
Cell In[41], line 3
      1 fig=plt.gcf()
      2 fig.set_size_inches(10,7)
----> 3 fig=sns.heatmap(iris.corr(),annot=True,cmap='cubehelix',linewidths=1,lin
color='k',square=True,mask=False, vmin=-1, vmax=1,cbar_kws={"orientation": "verti
cal"},cbar=True)

File ~\anaconda3\Lib\site-packages\pandas\core\frame.py:11049, in DataFrame.corr
(self, method, min_periods, numeric_only)
    11047 cols = data.columns
    11048 idx = cols.copy()
> 11049 mat = data.to_numpy(dtype=float, na_value=np.nan, copy=False)
    11051 if method == "pearson":
    11052     correl = libalgos.nancorr(mat, minp=min_periods)

File ~\anaconda3\Lib\site-packages\pandas\core\frame.py:1993, in DataFrame.to_num
py(self, dtype, copy, na_value)
    1991 if dtype is not None:
    1992     dtype = np.dtype(dtype)
-> 1993 result = self._mgr.as_array(dtype=dtype, copy=copy, na_value=na_value)
    1994 if result.dtype is not dtype:
    1995     result = np.asarray(result, dtype=dtype)

File ~\anaconda3\Lib\site-packages\pandas\core\internals\managers.py:1694, in Blo
ckManager.as_array(self, dtype, copy, na_value)
    1692     arr.flags.writeable = False
    1693 else:
-> 1694     arr = self._interleave(dtype=dtype, na_value=na_value)
    1695     # The underlying data was copied within _interleave, so no need
    1696     # to further copy if copy=True or setting na_value
    1698 if na_value is lib.no_default:

File ~\anaconda3\Lib\site-packages\pandas\core\internals\managers.py:1753, in Blo
ckManager._interleave(self, dtype, na_value)
    1751     else:
    1752         arr = blk.get_values(dtype)
-> 1753     result[rl.indexer] = arr
    1754     itemmask[rl.indexer] = 1
    1756 if not itemmask.all():

ValueError: could not convert string to float: 'Iris-setosa'

```

```

In [42]: fig=plt.gcf()
plt.figure(figsize=(10, 7))
fig=sns.heatmap(iris.corr(),annot=True,cmap='cubehelix',linewidths=1,linecolor='

```

```

-----
ValueError                                Traceback (most recent call last)
Cell In[42], line 3
      1 fig=plt.gcf()
      2 plt.figure(figsize=(10, 7))
----> 3 fig=sns.heatmap(iris.corr(),annot=True,cmap='cubehelix',linewidths=1,line
color='k',square=True,mask=False, vmin=-1, vmax=1,cbar_kws={"orientation": "verti
cal"},cbar=True)

File ~\anaconda3\Lib\site-packages\pandas\core\frame.py:11049, in DataFrame.corr
(self, method, min_periods, numeric_only)
    11047 cols = data.columns
    11048 idx = cols.copy()
> 11049 mat = data.to_numpy(dtype=float, na_value=np.nan, copy=False)
    11051 if method == "pearson":
    11052     correl = libalgos.nancorr(mat, minp=min_periods)

File ~\anaconda3\Lib\site-packages\pandas\core\frame.py:1993, in DataFrame.to_num
py(self, dtype, copy, na_value)
    1991 if dtype is not None:
    1992     dtype = np.dtype(dtype)
-> 1993 result = self._mgr.as_array(dtype=dtype, copy=copy, na_value=na_value)
    1994 if result.dtype is not dtype:
    1995     result = np.asarray(result, dtype=dtype)

File ~\anaconda3\Lib\site-packages\pandas\core\internals\managers.py:1694, in Blo
ckManager.as_array(self, dtype, copy, na_value)
    1692     arr.flags.writeable = False
    1693 else:
-> 1694     arr = self._interleave(dtype=dtype, na_value=na_value)
    1695     # The underlying data was copied within _interleave, so no need
    1696     # to further copy if copy=True or setting na_value
    1698 if na_value is lib.no_default:

File ~\anaconda3\Lib\site-packages\pandas\core\internals\managers.py:1753, in Blo
ckManager._interleave(self, dtype, na_value)
    1751     else:
    1752         arr = blk.get_values(dtype)
-> 1753     result[rl.indexer] = arr
    1754     itemmask[rl.indexer] = 1
    1756 if not itemmask.all():

ValueError: could not convert string to float: 'Iris-setosa'

```

```

In [43]: import seaborn as sns
import matplotlib.pyplot as plt

# Drop non-numeric column(s)
iris_numeric = iris.drop(columns=['Species'])

# Set figure size
plt.figure(figsize=(10, 7))

# Plot heatmap
sns.heatmap(
    iris_numeric.corr(),
    annot=True,
    cmap='cubehelix',
    linewidths=1,
    linecolor='k',

```

```

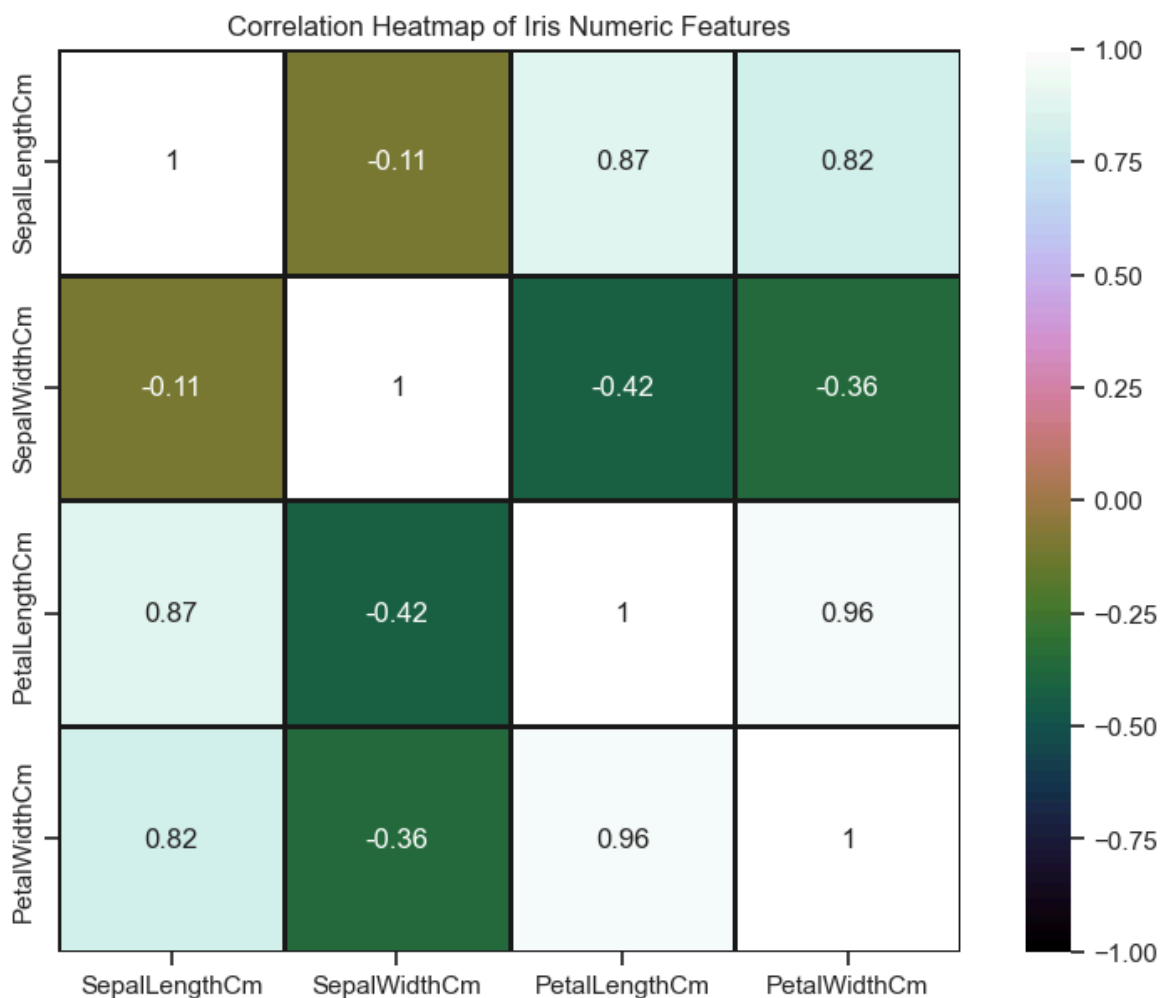
square=True,
vmin=-1,
vmax=1,
cbar_kws={"orientation": "vertical"},
cbar=True
)

plt.title("Correlation Heatmap of Iris Numeric Features")
plt.show()

```

<Figure size 1000x700 with 0 Axes>

<Figure size 1000x700 with 0 Axes>



Distribution Plot

```

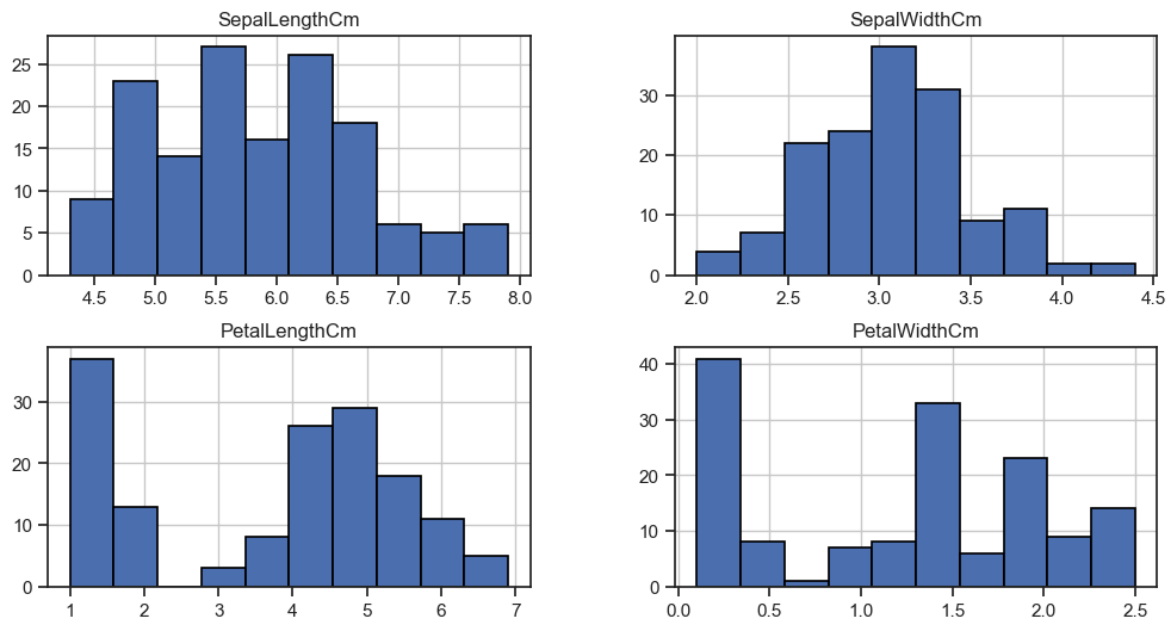
In [44]: iris.hist(edgecolor='black', linewidth=1.2)
fig=plt.gcf()
fig.set_size_inches(12,6)

```

```

In [45]: plt.show()

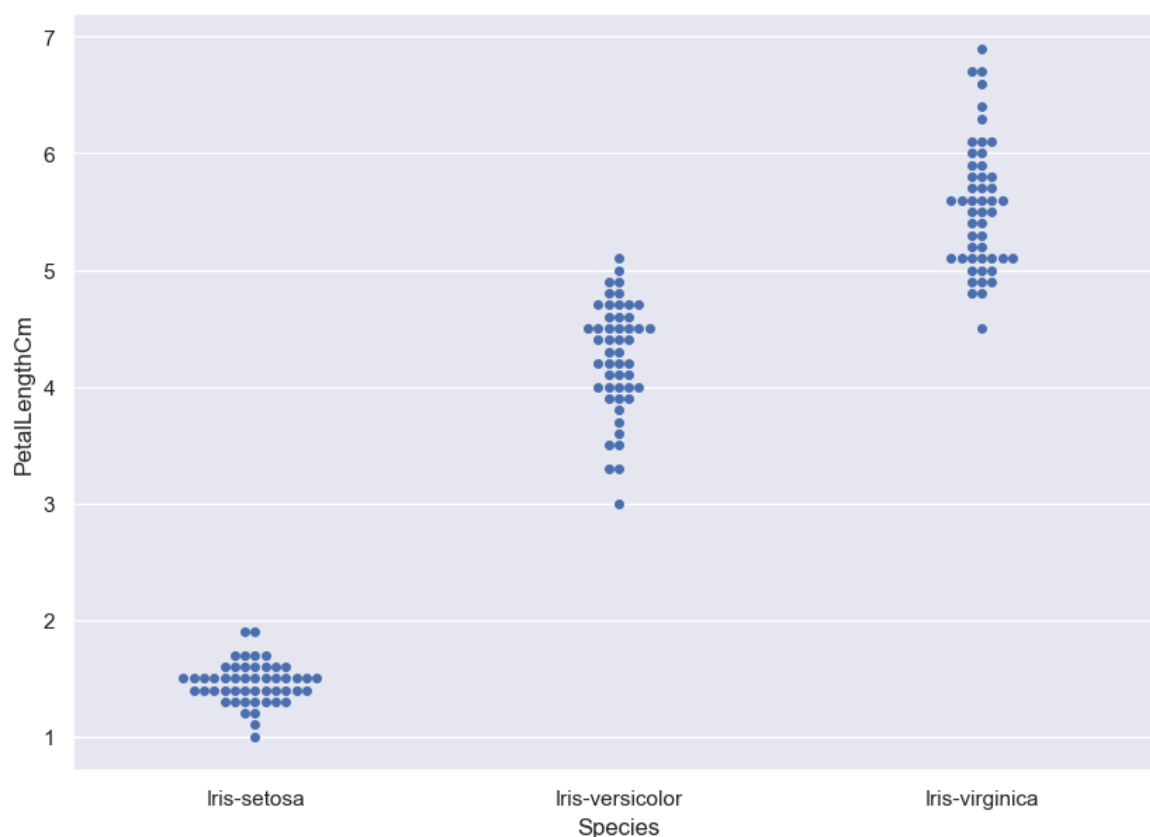
```



Swarm plot

```
In [46]: sns.set(style="darkgrid")
fig=plt.gcf()
fig.set_size_inches(10,7)
fig = sns.swarmplot(x="Species", y="PetalLengthCm", data=iris)
```

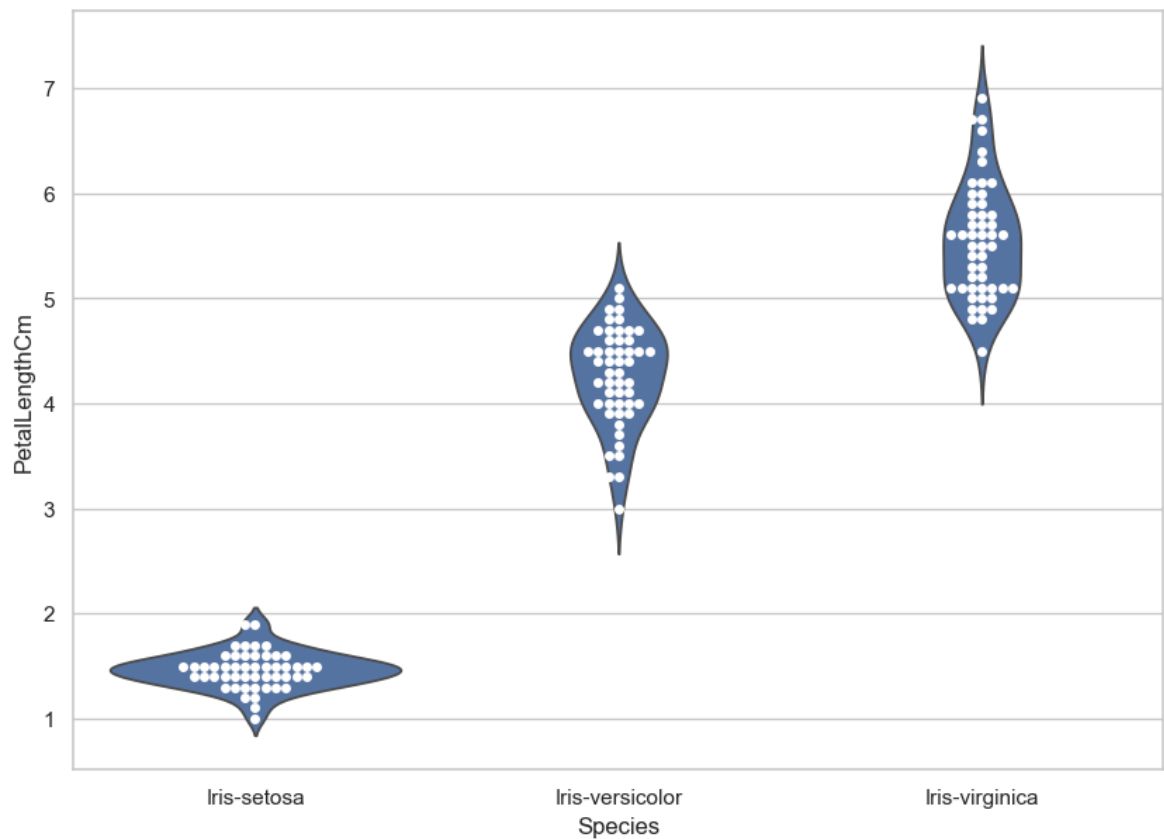
```
In [47]: plt.show()
```



```
In [48]: sns.set(style="whitegrid")
fig=plt.gcf()
fig.set_size_inches(10,7)
```

```
ax = sns.violinplot(x="Species", y="PetalLengthCm", data=iris, inner=None)  
ax = sns.swarmplot(x="Species", y="PetalLengthCm", data=iris, color="white", edge
```

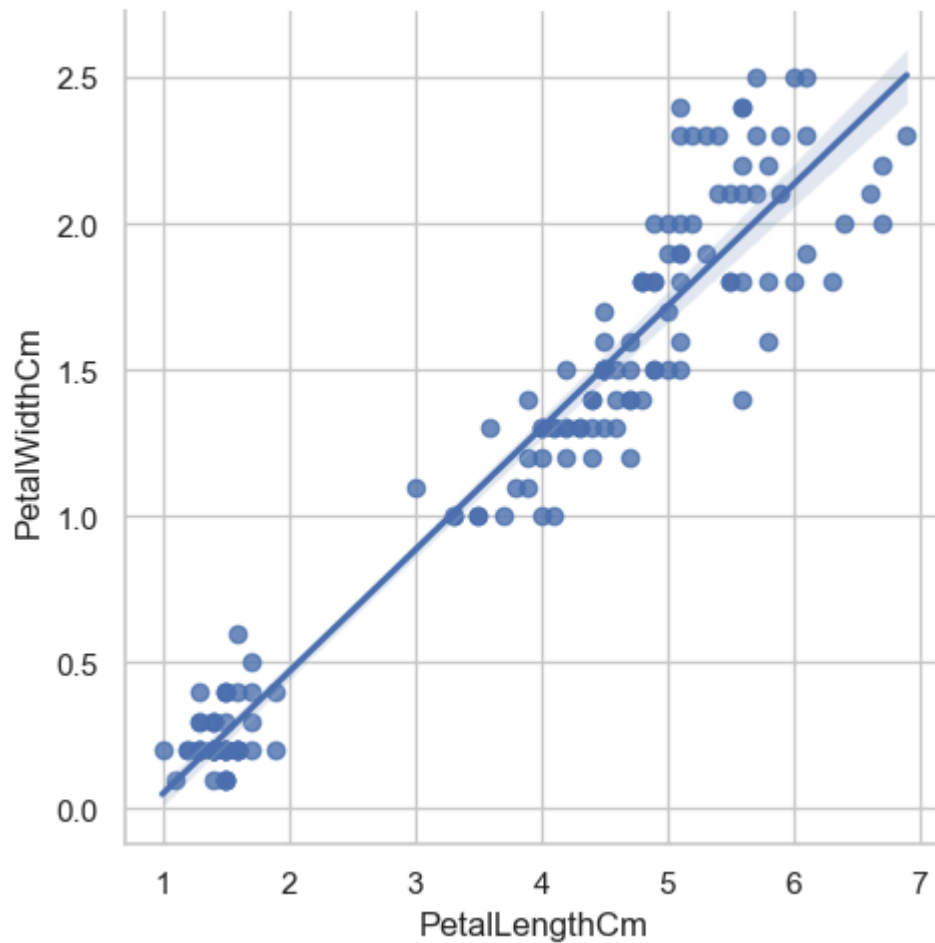
```
In [49]: plt.show()
```



LM plot

```
In [50]: fig=sns.lmplot(x="PetalLengthCm", y="PetalWidthCm",data=iris)
```

```
In [51]: plt.show()
```

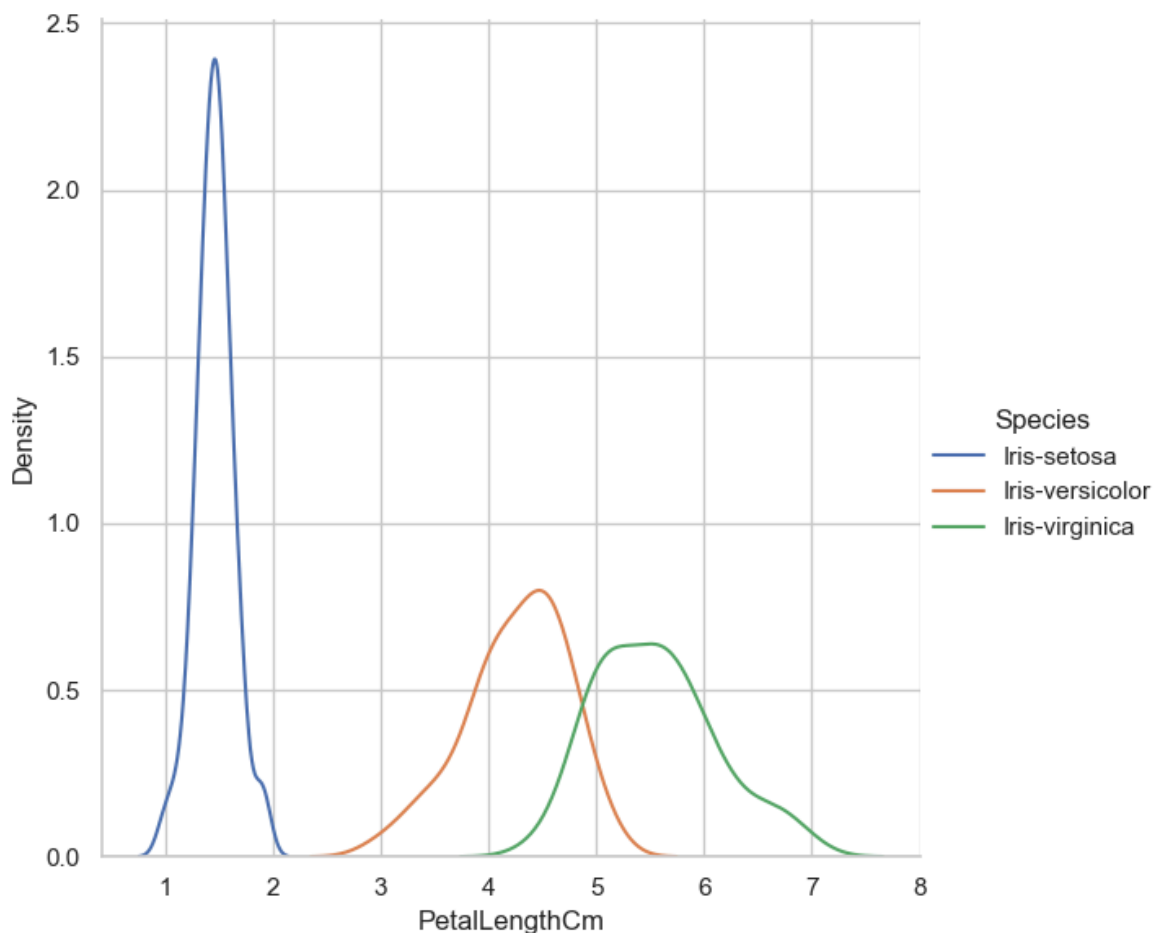


Facet Grid

```
In [53]: sns.FacetGrid(iris, hue="Species", height=6) \
        .map(sns.kdeplot, "PetalLengthCm") \
        .add_legend()
plt.ioff()
```

```
Out[53]: <contextlib.ExitStack at 0x1a83d0c1190>
```

```
In [54]: plt.show()
```



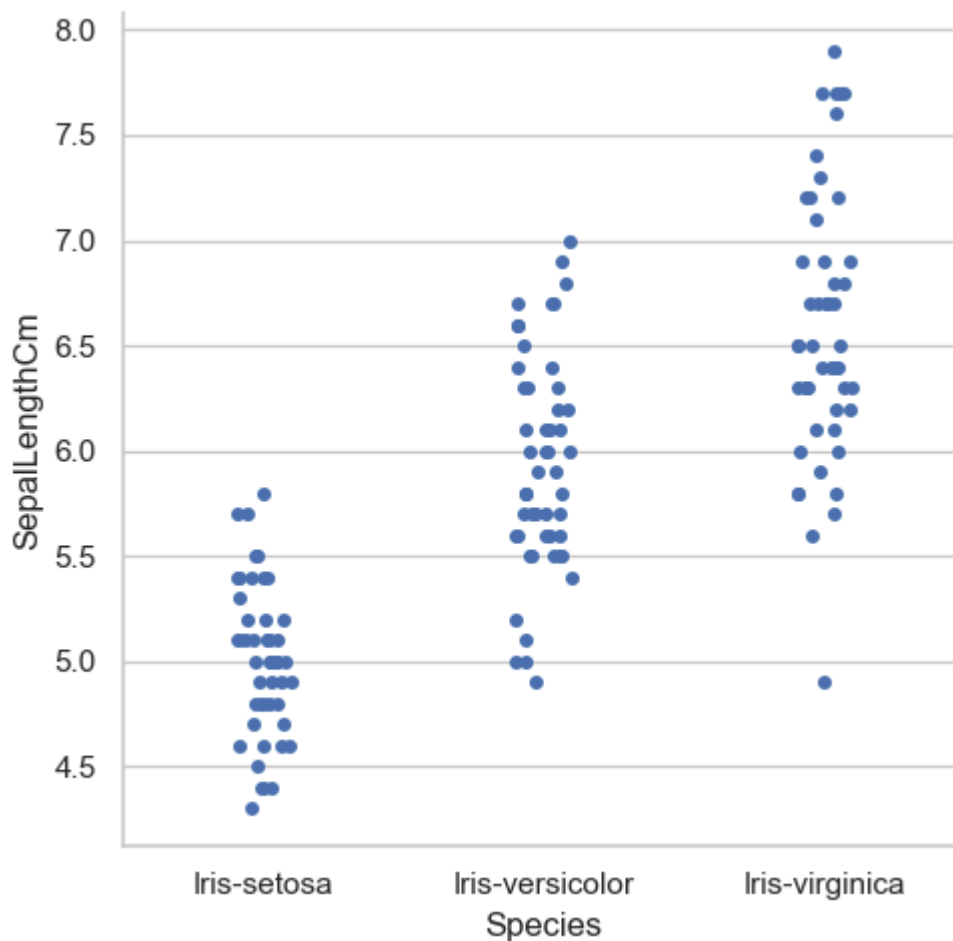
Factor plot

```
In [55]: sns.factorplot('Species', 'SepalLengthCm', data=iris)
plt.ioff()
plt.show()
```

```
-----
AttributeError                                Traceback (most recent call last)
Cell In[55], line 1
----> 1 sns.factorplot('Species', 'SepalLengthCm', data=iris)
      2 plt.ioff()
      3 plt.show()

AttributeError: module 'seaborn' has no attribute 'factorplot'
```

```
In [57]: sns.catplot(x='Species', y='SepalLengthCm', data=iris)
plt.ioff()
plt.show()
```

```
In [58]: sns.catplot(x='Species',y='SepalLengthCm',data=iris,ax=ax[0][0])
sns.catplot(x='Species',y='SepalLengthCm',data=iris,ax=ax[0][1])
sns.catplot(x='Species',y='SepalLengthCm',data=iris,ax=ax[1][0])
sns.catplot(x='Species',y='SepalLengthCm',data=iris,ax=ax[1][1])
```

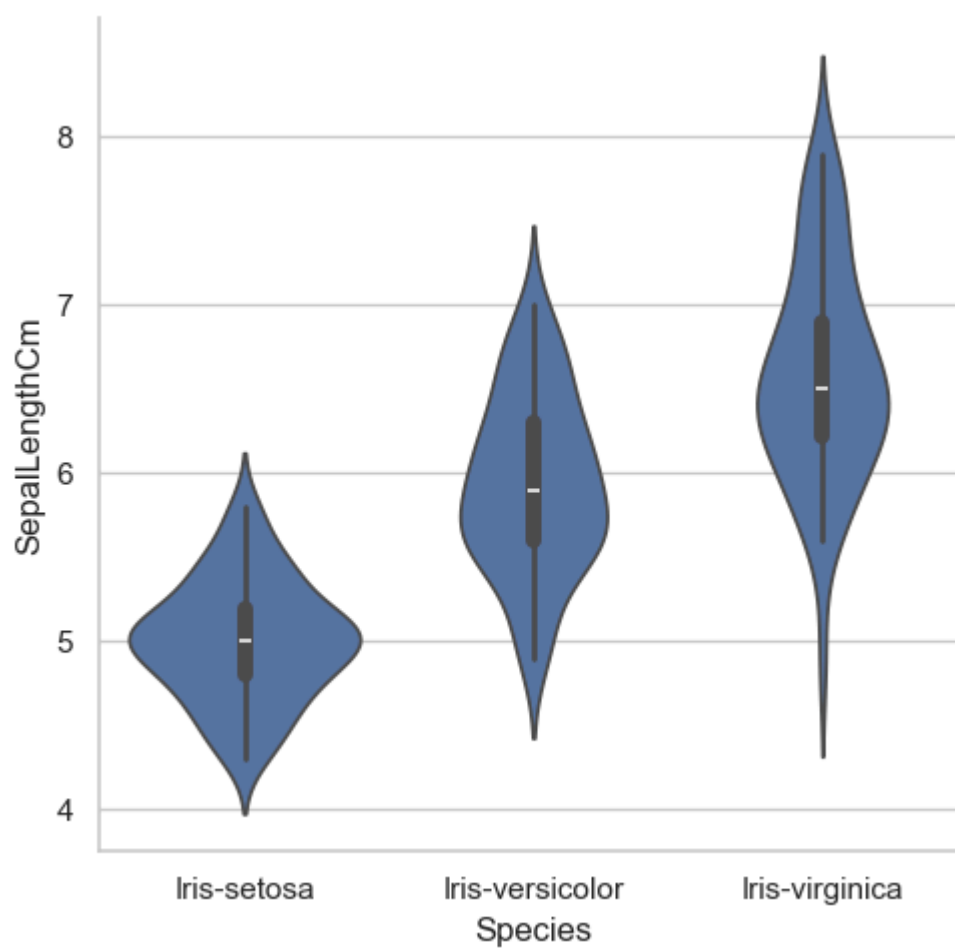
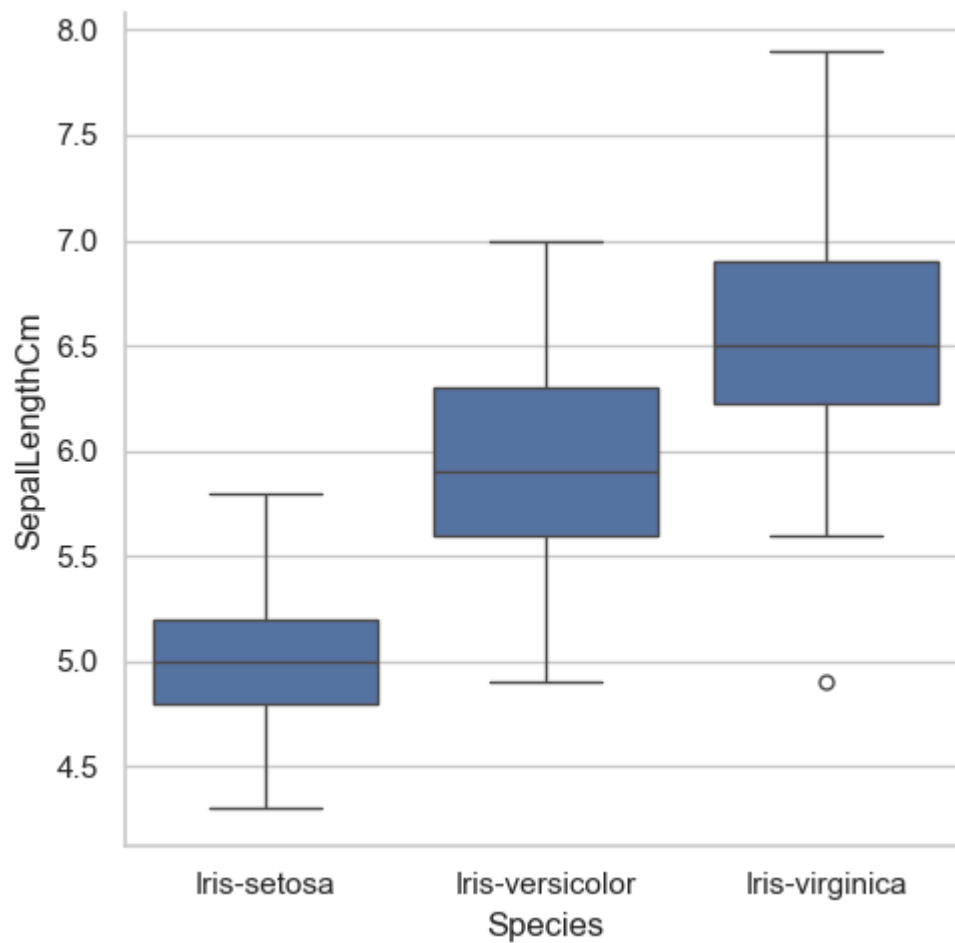
```
-----
TypeError                                Traceback (most recent call last)
Cell In[58], line 1
----> 1 sns.catplot(x='Species',y='SepalLengthCm',data=iris,ax=ax[0][0])
      2 sns.catplot(x='Species',y='SepalLengthCm',data=iris,ax=ax[0][1])
      3 sns.catplot(x='Species',y='SepalLengthCm',data=iris,ax=ax[1][0])

TypeError: 'Axes' object is not subscriptable
```

```
In [59]: sns.catplot(x='Species', y='SepalLengthCm', data=iris, kind='box')
sns.catplot(x='Species', y='SepalLengthCm', data=iris, kind='violin')
```

```
Out[59]: <seaborn.axisgrid.FacetGrid at 0x1a83d5095b0>
```

```
In [60]: plt.show()
```



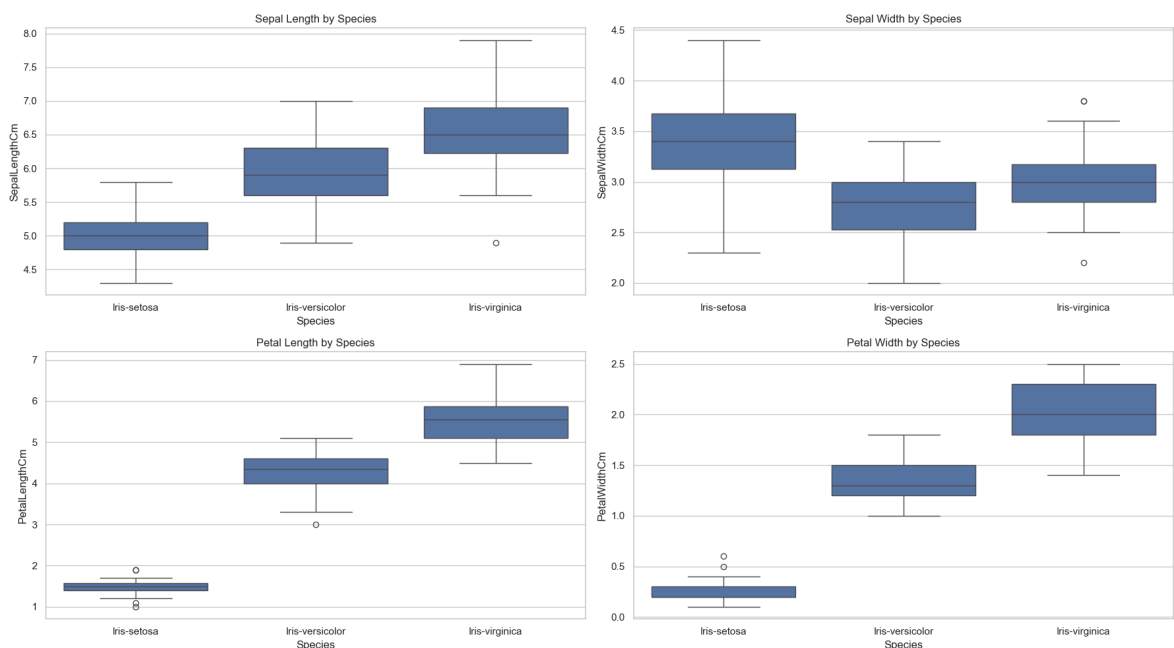
```
In [61]: import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Create a 2x2 grid of subplots
fig, ax = plt.subplots(2, 2, figsize=(18, 10))

# Plotting each feature using Seaborn boxplot
sns.boxplot(x='Species', y='SepalLengthCm', data=iris, ax=ax[0][0])
sns.boxplot(x='Species', y='SepalWidthCm', data=iris, ax=ax[0][1])
sns.boxplot(x='Species', y='PetalLengthCm', data=iris, ax=ax[1][0])
sns.boxplot(x='Species', y='PetalWidthCm', data=iris, ax=ax[1][1])

# Optional: Add titles to each subplot
ax[0][0].set_title('Sepal Length by Species')
ax[0][1].set_title('Sepal Width by Species')
ax[1][0].set_title('Petal Length by Species')
ax[1][1].set_title('Petal Width by Species')

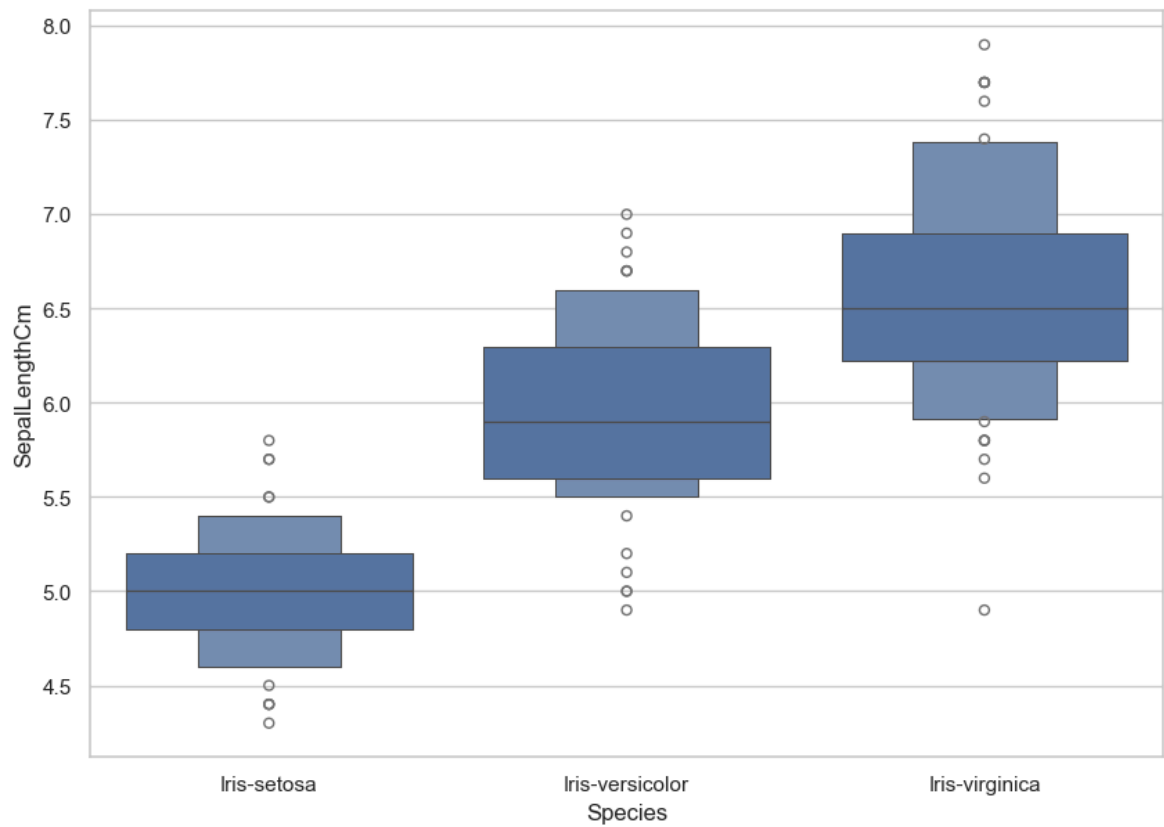
plt.tight_layout()
plt.show()
```



Boxen plot

```
In [62]: fig=plt.gcf()
fig.set_size_inches(10,7)
fig=sns.boxenplot(x='Species',y='SepalLengthCm',data=iris)
```

```
In [63]: plt.show()
```



KDE plot

```
In [64]: sub=iris[iris['Species']=='Iris-setosa']
sns.kdeplot(data=sub[['SepalLengthCm', 'SepalWidthCm']], cmap="plasma", shade=True)
plt.title('Iris-setosa')
plt.xlabel('Sepal Length Cm')
plt.ylabel('Sepal Width Cm')
```

```

-----
AttributeError                                Traceback (most recent call last)
Cell In[64], line 2
      1 sub=iris[iris['Species']=='Iris-setosa']
----> 2 sns.kdeplot(data=sub[['SepalLengthCm', 'SepalWidthCm']], cmap="plasma", sha
de=True, shade_lowest=False)
      3 plt.title('Iris-setosa')
      4 plt.xlabel('Sepal Length Cm')

File ~\anaconda3\Lib\site-packages\seaborn\distributions.py:1701, in kdeplot(data, x, y, hue, weights, palette, hue_order, hue_norm, color, fill, multiple, common_norm, common_grid, cumulative, bw_method, bw_adjust, warn_singular, log_scale, levels, thresh, gridsize, cut, clip, legend, cbar, cbar_ax, cbar_kws, ax, **kwargs)
    1697 if p.univariate:
    1699     plot_kws = kwargs.copy()
-> 1701     p.plot_univariate_density(
    1702         multiple=multiple,
    1703         common_norm=common_norm,
    1704         common_grid=common_grid,
    1705         fill=fill,
    1706         color=color,
    1707         legend=legend,
    1708         warn_singular=warn_singular,
    1709         estimate_kws=estimate_kws,
    1710         **plot_kws,
    1711     )
    1713 else:
    1715     p.plot_bivariate_density(
    1716         common_norm=common_norm,
    1717         fill=fill,
    (...)
    1727         **kwargs,
    1728     )

File ~\anaconda3\Lib\site-packages\seaborn\distributions.py:1024, in _DistributionPlotter.plot_univariate_density(self, multiple, common_norm, common_grid, warn_singular, fill, color, legend, estimate_kws, **plot_kws)
    1021     artist = partial(mpl.lines.Line2D, [], [])
    1023 ax_obj = self.ax if self.ax is not None else self.facets
-> 1024 self._add_legend(
    1025     ax_obj, artist, fill, False, multiple, alpha, plot_kws, {},
    1026 )

File ~\anaconda3\Lib\site-packages\seaborn\distributions.py:156, in _DistributionPlotter._add_legend(self, ax_obj, artist, fill, element, multiple, alpha, artist_kws, legend_kws)
    153     if "facecolor" in kws:
    154         kws.pop("color", None)
--> 156     handles.append(artist(**kws))
    157     labels.append(level)
    159 if isinstance(ax_obj, mpl.axes.Axes):

File ~\anaconda3\Lib\site-packages\matplotlib\patches.py:98, in Patch.__init__(self, edgecolor, facecolor, color, linewidth, linestyle, antialiased, hatch, fill, capstyle, joinstyle, **kwargs)
    95 self.set_joinstyle(joinstyle)
    97 if len(kwargs):
--> 98     self._internal_update(kwargs)

```

```

File ~\anaconda3\Lib\site-packages\matplotlib\artist.py:1216, in Artist._internal_update(self, kwargs)
    1209 def _internal_update(self, kwargs):
    1210     """
    1211     Update artist properties without prenormalizing them, but generating
    1212     errors as if calling `set`.
    1213
    1214     The lack of prenormalization is to maintain backcompatibility.
    1215     """
-> 1216     return self._update_props(
    1217         kwargs, "{cls.__name__}.set() got an unexpected keyword argument
    1218         "{prop_name!r}")

File ~\anaconda3\Lib\site-packages\matplotlib\artist.py:1190, in Artist._update_props(self, props, errfmt)
    1188         func = getattr(self, f"set_{k}", None)
    1189         if not callable(func):
-> 1190             raise AttributeError(
    1191                 errfmt.format(cls=type(self), prop_name=k))
    1192         ret.append(func(v))
    1193 if ret:

AttributeError: Patch.set() got an unexpected keyword argument 'cmap'

```

```

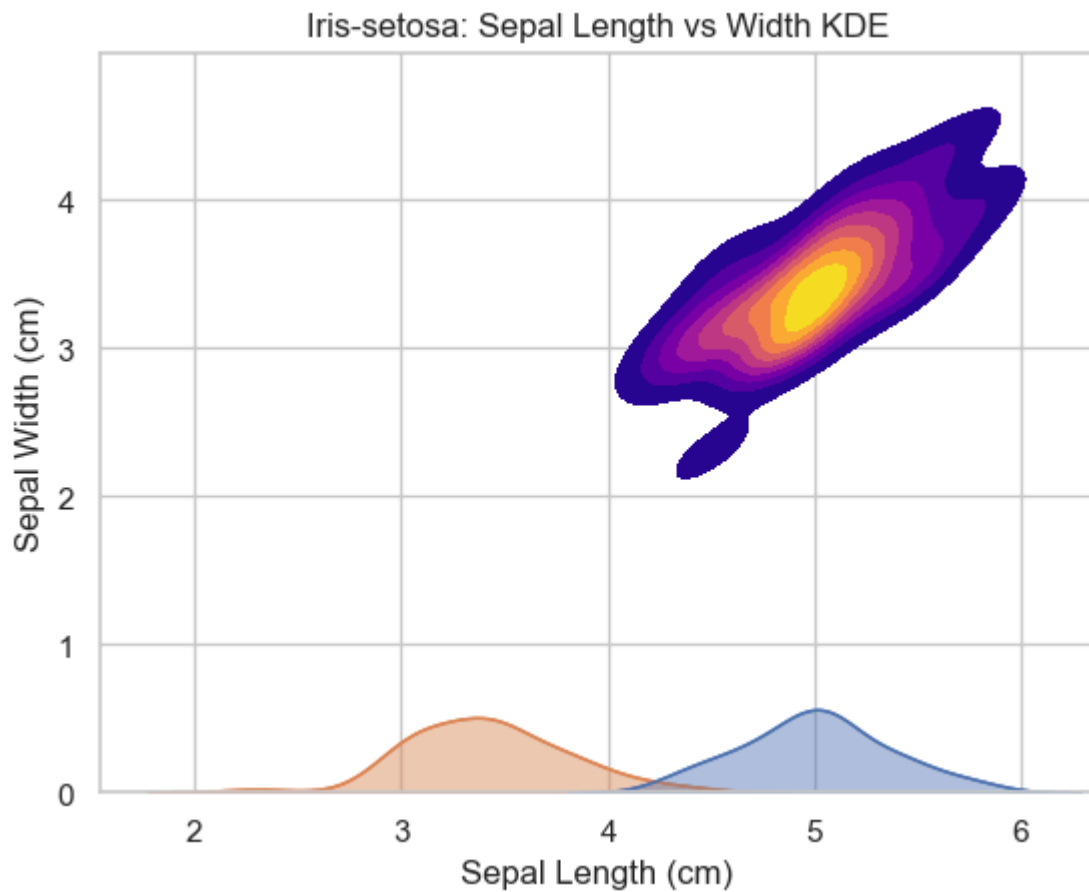
In [66]: import seaborn as sns
import matplotlib.pyplot as plt

# Filter only Iris-setosa
sub = iris[iris['Species'] == 'Iris-setosa']

# Plot 2D KDE
sns.kdeplot(
    x=sub['SepalLengthCm'],
    y=sub['SepalWidthCm'],
    cmap="plasma",
    fill=True, # replaces deprecated `shade=True`
    thresh=0.05 # optionally control density threshold
)

plt.title('Iris-setosa: Sepal Length vs Width KDE')
plt.xlabel('Sepal Length (cm)')
plt.ylabel('Sepal Width (cm)')
plt.show()

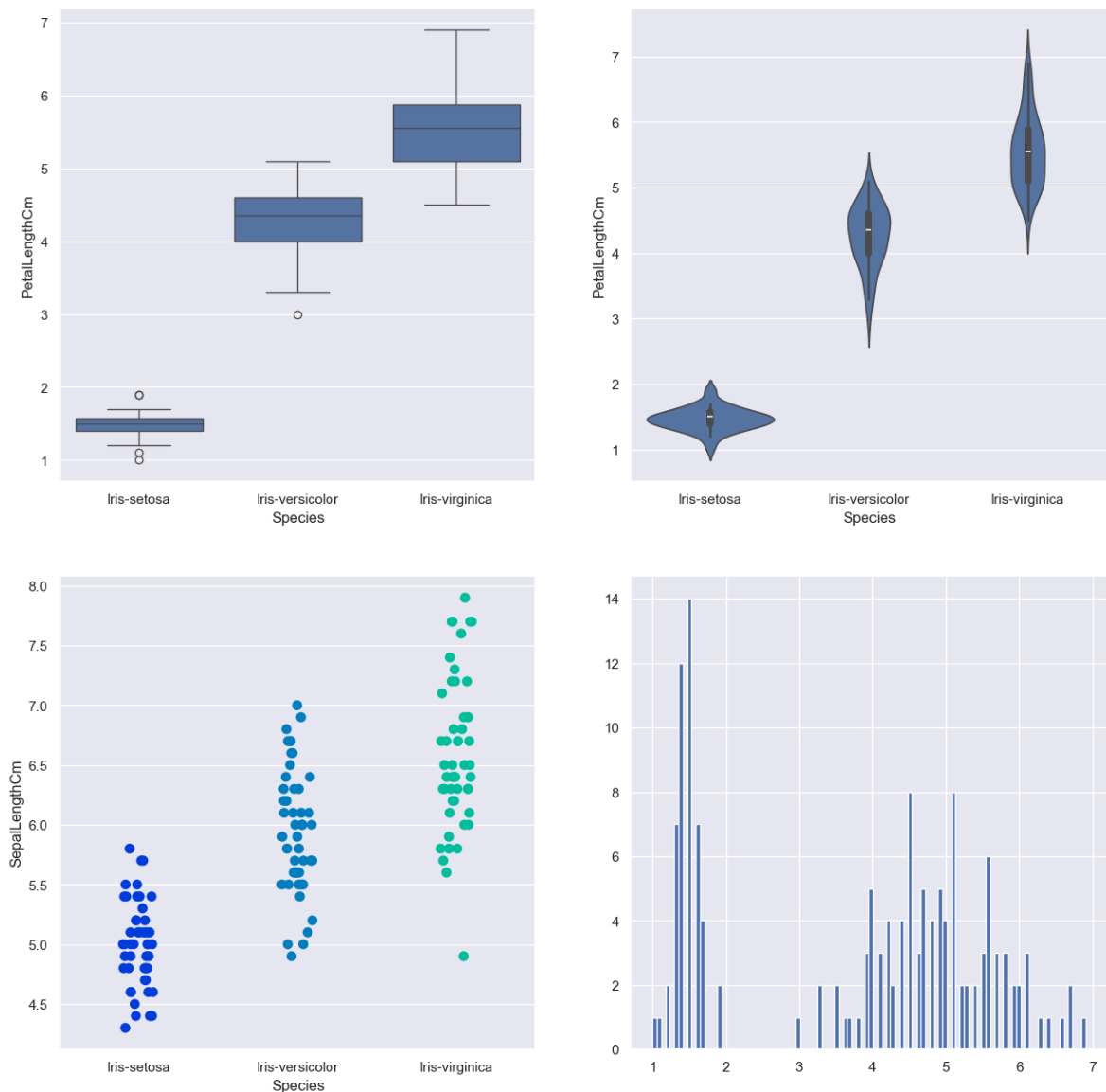
```



Dashboard

```
In [67]: sns.set_style('darkgrid')
f, axes = plt.subplots(2, 2, figsize=(15, 15))

k1 = sns.boxplot(x="Species", y="PetalLengthCm", data=iris, ax=axes[0, 0])
k2 = sns.violinplot(x='Species', y='PetalLengthCm', data=iris, ax=axes[0, 1])
k3 = sns.stripplot(x='Species', y='SepalLengthCm', data=iris, jitter=True, edgecolor='
#axes[1, 1].hist(iris.hist, bin=10)
axes[1, 1].hist(iris.PetalLengthCm, bins=100)
#k2.set(xlim=(-1, 0.8))
plt.show()
```



stacked Histogram

```
In [68]: iris['Species'] = iris['Species'].astype('category')
```

```
In [69]: iris.head()
```

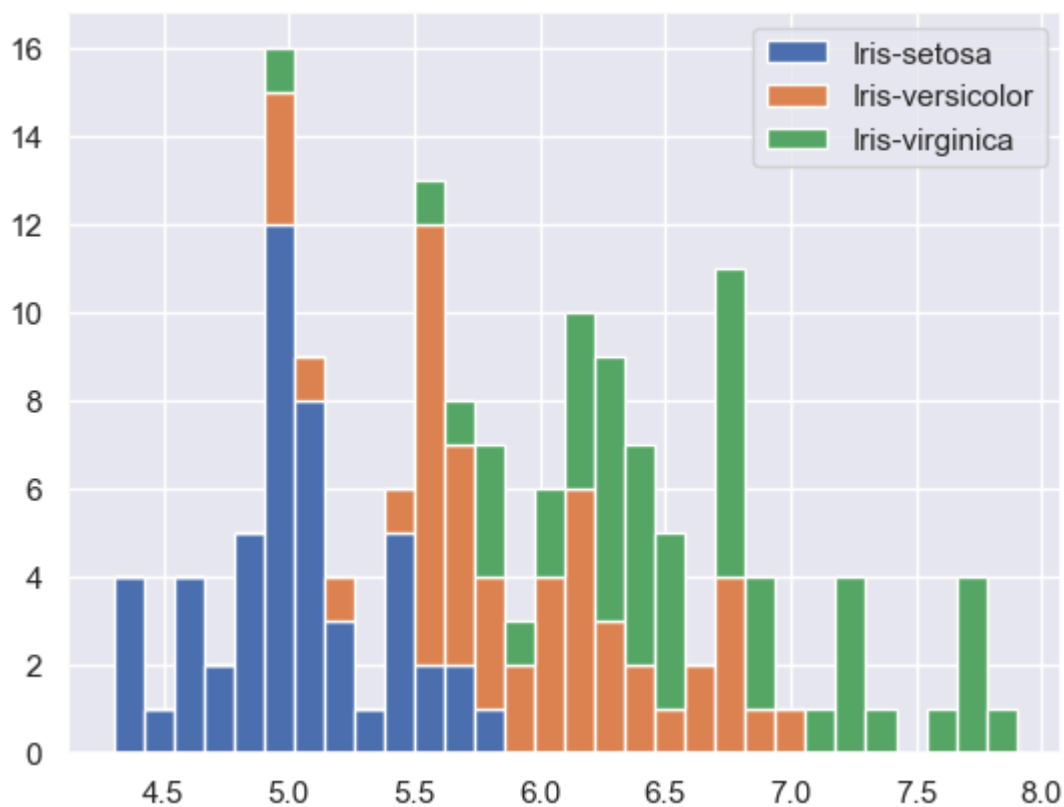
```
Out[69]:
```

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

```
In [70]: list1=list()
mylabels=list()
for gen in iris.Species.cat.categories:
    list1.append(iris[iris.Species==gen].SepalLengthCm)
    mylabels.append(gen)
```



```
h=plt.hist(list1,bins=30,stacked=True,rwidth=1,label=mylabels)
plt.legend()
plt.show()
```



Area plot

```
In [71]: iris['SepalLengthCm'] = iris['SepalLengthCm'].astype('category')
```

```
In [72]: iris.head()
```

```
Out[72]:
```

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

```
In [77]: iris.plot.area(y='SepalLengthCm',alpha=0.4,figsize=(12, 6));
select_dtypes(include='number')
iris.plot.area(y=['SepalLengthCm','SepalWidthCm','PetalLengthCm','PetalWidthCm'])
```

```

-----
TypeError                                Traceback (most recent call last)
Cell In[77], line 1
----> 1 iris.plot.area(y='SepalLengthCm',alpha=0.4,figsize=(12, 6));
      2 select_dtypes(include='number')
      3 iris.plot.area(y=['SepalLengthCm','SepalWidthCm','PetalLengthCm','PetalWi
dthCm'],alpha=0.4,figsize=(12, 6))

File ~\anaconda3\Lib\site-packages\pandas\plotting\_core.py:1603, in PlotAccesso
r.area(self, x, y, stacked, **kwargs)
    1526 def area(
    1527     self,
    1528     x: Hashable | None = None,
    (...)
    1531     **kwargs,
    1532 ) -> PlotAccessor:
    1533     """
    1534     Draw a stacked area plot.
    1535     (...)
    1601         >>> ax = df.plot.area(x='day')
    1602         """
-> 1603     return self(kind="area", x=x, y=y, stacked=stacked, **kwargs)

File ~\anaconda3\Lib\site-packages\pandas\plotting\_core.py:1030, in PlotAccesso
r.__call__(self, *args, **kwargs)
    1027         label_name = label_kw or data.columns
    1028         data.columns = label_name
-> 1030 return plot_backend.plot(data, kind=kind, **kwargs)

File ~\anaconda3\Lib\site-packages\pandas\plotting\_matplotlib\_init__.py:70, in
plot(data, kind, **kwargs)
     68         ax = plt.gca()
     69         kwargs["ax"] = getattr(ax, "left_ax", ax)
---> 70 plot_obj = PLOT_CLASSES[kind](data, **kwargs)
     71 plot_obj.generate()
     72 plot_obj.draw()

File ~\anaconda3\Lib\site-packages\pandas\plotting\_matplotlib\core.py:1728, in A
reaPlot.__init__(self, data, **kwargs)
    1722 with warnings.catch_warnings():
    1723     warnings.filterwarnings(
    1724         "ignore",
    1725         "Downcasting object dtype arrays",
    1726         category=FutureWarning,
    1727     )
-> 1728     data = data.fillna(value=0)
    1729 LinePlot.__init__(self, data, **kwargs)
    1731 if not self.stacked:
    1732     # use smaller alpha to distinguish overlap

File ~\anaconda3\Lib\site-packages\pandas\core\generic.py:7349, in NDFrame.fillna
(self, value, method, axis, inplace, limit, downcast)
    7342     else:
    7343         raise TypeError(
    7344             '"value" parameter must be a scalar, dict '
    7345             '"or Series, but you passed a "'
    7346             f'{type(value).__name__}'
    7347         )
-> 7349     new_data = self._mgr.fillna(

```

```

7350         value=value, limit=limit, inplace=inplace, downcast=downcast
7351     )
7353 elif isinstance(value, (dict, ABCSeries)):
7354     if axis == 1:

File ~\anaconda3\Lib\site-packages\pandas\core\internals\base.py:186, in DataManager.fillna(self, value, limit, inplace, downcast)
    182 if limit is not None:
    183     # Do this validation even if we go through one of the no-op paths
    184     limit = libalgos.validate_limit(None, limit=limit)
--> 186 return self.apply_with_block(
    187     "fillna",
    188     value=value,
    189     limit=limit,
    190     inplace=inplace,
    191     downcast=downcast,
    192     using_cow=using_copy_on_write(),
    193     already_warned=_AlreadyWarned(),
    194 )

File ~\anaconda3\Lib\site-packages\pandas\core\internals\managers.py:363, in BaseBlockManager.apply(self, f, align_keys, **kwargs)
    361     applied = b.apply(f, **kwargs)
    362     else:
--> 363     applied = getattr(b, f)(**kwargs)
    364     result_blocks = extend_blocks(applied, result_blocks)
    366 out = type(self).from_blocks(result_blocks, self.axes)

File ~\anaconda3\Lib\site-packages\pandas\core\internals\blocks.py:2334, in ExtensionBlock.fillna(self, value, limit, inplace, downcast, using_cow, already_warned)
    2331 except TypeError:
    2332     # 3rd party EA that has not implemented copy keyword yet
    2333     refs = None
-> 2334     new_values = self.values.fillna(value=value, method=None, limit=limit)
    2335     # issue the warning *after* retrying, in case the TypeError
    2336     # was caused by an invalid fill_value
    2337     warnings.warn(
    2338         # GH#53278
    2339         "ExtensionArray.fillna added a 'copy' keyword in pandas "
    2340         (...)
    2345         stacklevel=find_stack_level(),
    2346     )

File ~\anaconda3\Lib\site-packages\pandas\core\arrays\_mixins.py:376, in NDArrayBackedExtensionArray.fillna(self, value, method, limit, copy)
    373 else:
    374     # We validate the fill_value even if there is nothing to fill
    375     if value is not None:
--> 376         self._validate_setitem_value(value)
    378     if not copy:
    379         new_values = self[:]

File ~\anaconda3\Lib\site-packages\pandas\core\arrays\categorical.py:1589, in Categorical._validate_setitem_value(self, value)
    1587     return self._validate_listlike(value)
    1588 else:
-> 1589     return self._validate_scalar(value)

```

```
File ~\anaconda3\Lib\site-packages\pandas\core\arrays\categorical.py:1614, in Categorical._validate_scalar(self, fill_value)
    1612     fill_value = self._unbox_scalar(fill_value)
    1613 else:
-> 1614     raise TypeError(
    1615         "Cannot setitem on a Categorical with a new "
    1616         f"category ({fill_value}), set the categories first"
    1617     ) from None
    1618 return fill_value
```

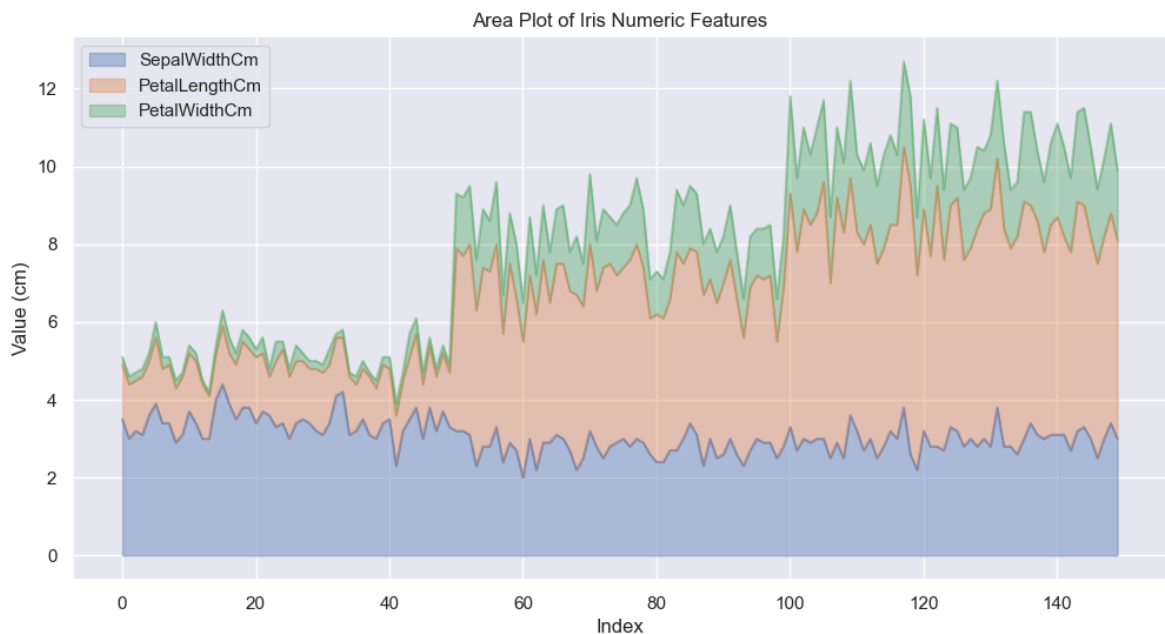
TypeError: Cannot setitem on a Categorical with a new category (0), set the categories first

```
In [78]: import matplotlib.pyplot as plt

# Select only numeric columns (this automatically excludes 'Species')
iris_numeric = iris.select_dtypes(include='number')

# Plot area chart
iris_numeric.plot.area(alpha=0.4, figsize=(12, 6))

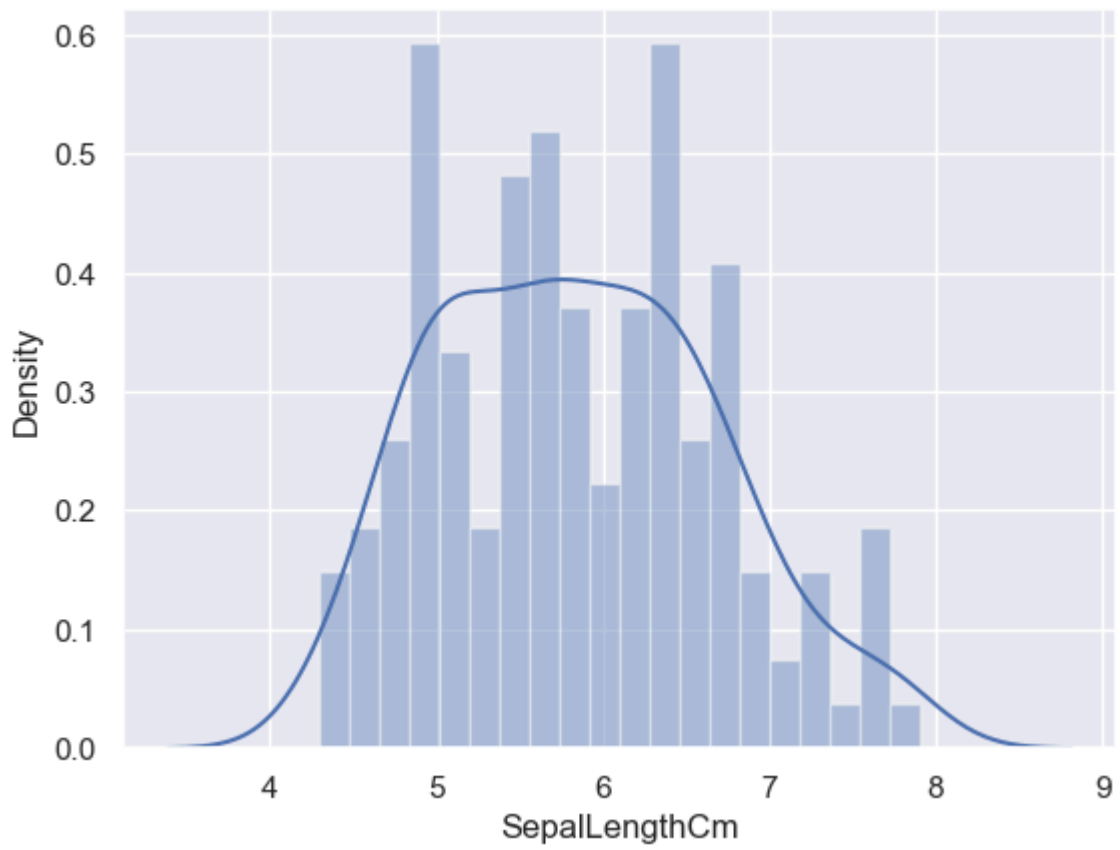
plt.title('Area Plot of Iris Numeric Features')
plt.ylabel('Value (cm)')
plt.xlabel('Index')
plt.show()
```



Dist plot

```
In [79]: sns.distplot(iris['SepalLengthCm'], kde=True, bins=20);
```

```
In [80]: plt.show()
```



In []: